

 Complete Zero to Hero Notes

# Handwritten-Style Backend Revision Guide

Designed for crystal clear conceptual understanding. Every single topic has Indian real-life examples, visual arrow-based code anatomies, complete working code, and line-by-line mechanics!

## Chapter 01

 Core JS

### JavaScript Essentials & ES6 Mechanics

Variable scopes, TDZ, Destructuring, Rest & Spread, Array methods (map/filter/reduce), and Callbacks to Async/Await.

TDZ & Scopes

Arrow Anatomy

Promises

Async/Await

## Chapter 02

 Architecture

### Node Architecture & Event Loop Internals

V8 Ignition/TurboFan, Libuv 4 C-threads, 6 Event Loop Phases, Microtask Queue vs nextTick, CommonJS Module IIFE wrapper.

V8 + Libuv

6 Loop Phases

process.nextTick

Module IIFE

### Chapter 03

 I/O Engine

## *File System & Streams Mastery*

Sync vs Async Non-Blocking I/O, Buffer memory allocation, 4 Stream types, Backpressure flow control & .pipe().

fs.readFile

Buffer.alloc

Streams

Backpressure

### Chapter 04

 Raw HTTP

## *Raw Node.js HTTP Server*

http.createServer lifecycle, req as readable stream chunks, res headers, manual URL routing & POST data assembly.

http.createServer

Stream Chunks

Manual Routing

### Chapter 05

 Framework

## *Express.js Fundamentals*

express() factory function, HTTP verb abstractions, res.send() auto-content-type vs res.json(), server bootstrapping.

express() App

HTTP Verbs

res.send vs json

### Chapter 06

 Middleware

## *Middleware Architecture & Pipeline*

(req, res, next) pipeline, next() execution pointer, 5 middleware types, 4-argument error handler arity mechanics.

(req, res, next)

5 Types

Error Middleware (4 args)

### Chapter 07

 Payload Data

## *Request Data & Body Parsing*

Extracting route params (req.params), query strings (req.query), body parsers (express.urlencoded, express.json).

req.params

req.query

express.urlencoded

express.json

### Chapter 08

 REST Routing

## *Dynamic Routing & REST APIs*

Route parameters, route precedence (/new before /:id), REST conventions, full in-memory CRUD implementation.

Route Precedence

REST Status Codes

In-Memory CRUD

## Chapter 09

 Architecture

# MVC Pattern & Modular Routers

`express.Router()` sub-applications, Model-View-Controller separation of concerns, Controller exports, scalable structure.

`express.Router()`

MVC Flow

Controller Decoupling

## Chapter 10

 SSR Views

# Static Assets & EJS Templating

`express.static()` asset serving, EJS view engine setup, `<%= %>` vs `<% %>` vs `<%- %>`, server-side rendering with partials.

`express.static()`

EJS Syntax Tags

Partials Inclusion

## Chapter 11

 Production

# Production-Ready Node.js

`cors`, `helmet` security headers, Global Error Handler, `nodemon` workflow, production `app.js` template.

`cors`

`helmet`

Error Handler

## *JavaScript Essentials & ES6 Mechanics*

---

Topics Explanation with Indian Examples

## 1. Variable Scopes & Temporal Dead Zone (TDZ)

### 🧐 1.1 Problem Kya Thi? (Why ES6 Introduced let & const)

ES5 tak JavaScript me sirf `var` keyword tha. `var` ke saath do bohot khatarnak problems thi:

1. **Accidental Re-declaration:** Agar 500 lines ke code me galti se dubara `var user = "amit"` likh doge, toh JavaScript koi error throw nahi karta aur purana data overwrite kar deta hai.
2. **No Block Scope:** `var` curly braces `{ ... }` ko respect nahi karta. Loop ke baad bhi variable bahar zinda reh kar memory pollute karta hai!

snippet\_var\_vs\_let.js

Scope Problem Example

```
// ⚠️ Problem with var: No block scope & silent overwrite!
var role = "Junior Dev";
if (true) {
  var role = "Team Lead"; // Overwrites outer variable silently!
}
console.log(role); // Outputs: "Team Lead" (Outer variable corrupted!)

// ✅ Solution with let: Strict Block Scope!
let score = 50;
if (true) {
  let score = 99; // Lives only inside this { ... } block
}
console.log(score); // Outputs: 50 (Outer variable stays completely safe!)
```

### 💡 1.2 Hoisting & Temporal Dead Zone (TDZ) Explained with Code

- **Hoisting:** JavaScript code execute karne se pehle ek *Memory Creation Phase* chalata hai jisme saare variables aur functions ko scope ke top par memory allocate hoti hai. `var` ko `undefined` se initialize kar diya jata hai, jabki `let` aur `const` ko memory toh milti hai lekin unhe `<uninitialized>` chhod diya jata hai.
- **Temporal Dead Zone (TDZ):** Scope me enter hone se lekar `let/const` ke actual declaration line tak ka time period TDZ kehlata hai. Is doran

variable access karne par JS turant `ReferenceError` phakta hai  
taaki bug execute hi na ho!

snippet\_tdz\_demo.js

TDZ Behavior

```
// ⚠️ var Hoisting gives undefined (Silent bug)
console.log(city); // Output: undefined
var city = "Mumbai";

// 🚫 let / const inside TDZ throws error immediately!
// ⚡ TDZ for 'state' starts here
console.log(state); // ⚡ ReferenceError: Cannot access 'state' before initializatio
let state = "Maharashtra"; // ➡️ TDZ ends here!
```



### *Indian Real-Life Analogy: Society Notice Board vs Biometric Flat*

- **var**: Society ka purana khula notice board — koi bhi aakar naya notice chipka kar purana overwrite kar deta hai (Silent Global Bug).
- **let / const**: Har resident ka personal flat with biometric lock — jab tak aap flat ke door `{ }` ke andar sahi line pe unlock nahi karte, bahar se koi touch nahi kar sakta (Block Scope & TDZ Protection).



# Visual Code Anatomy: Variable Lifecycle & TDZ Memory Flow

MEMORY  
HOISTING FLOW

**var a;** —————> Memory Allocated = undefined (Hoisted & Initialized)

**let b;** —————> Memory Allocated = **<Uninitialized>** (🛑 TDZ Active!)

// --- Execution Phase ---

console.log(a); —————> Prints: **undefined** (Silent Bug!)

console.log(b); —————> ✨ **ReferenceError: Cannot access 'b' before initialization**

**let b = 20;** —————> ✅ **TDZ Ends!** Variable is now safe.

🛑 **TDZ Zone**

**Temporal Dead Zone:** Scope me enter hone se lekar declaration line tak memory locked rehti hai.

✅ **Safety Net**

**Why it exists:** Buggy uninitialized variables ko galti se padhne se rokta hai!

## 2. Callbacks, Promises & Async/Await (Zero to Hero)

### 🤔 2.1 Async Code Kyu Zaroori Hai? (The Synchronous Blocking Problem)

JavaScript **Single-Threaded** hai — iske paas sirf ek Call Stack hota hai. Matlab ek time par sirf ek hi line execute ho sakti hai.

Agar aap database se 50,000 records fetch karne ka kaam synchronous (blocking) karoge, toh jab tak DB se data nahi aayega (2 seconds), pura server freeze ho jayega aur baki 10,000 users ki requests line me atki rahengi!

snippet\_blocking\_vs\_nonblocking.js

Synchronous Blocking Freeze

```
// ⚠️ BAD (Synchronous Blocking): Freezes the whole Node.js process!
console.log("User 1 requests profile");
// Simulating a heavy 3-second blocking task:
const freezeTill = Date.now() + 3000;
while (Date.now() < freezeTill) { /* CPU is blocked 100% */ }
console.log("User 1 profile loaded!");
console.log("User 2 request finally reached..."); // Waited 3 seconds unnecessarily!
```

### 📞 2.2 Callback Kya Hai Aur Callback Hell Kyu Hua?

**Callback:** Ek function ko dusre function me as an argument pass karna taaki jab async task (DB fetch / Timer) complete ho, toh wo function execute ho sake.

**Callback Hell (Pyramid of Doom):** Jab ek async kaam ke baad dusra, fir teesra kaam karna padta hai, toh code right side me triangle jaisa banne lagta hai (Inversion of Control & Hard to Debug).



```
// ⚠️ Callback Hell: Deep nesting makes error handling a nightmare!
fetchUser(101, function(user) {
  fetchUserCart(user.id, function(cart) {
    calculateBill(cart.items, function(bill) {
      processPayment(bill.amount, function(receipt) {
        sendInvoiceEmail(receipt.id, function(status) {
          console.log("Done! But look at the horrible triangle nesting! 😱");
        });
      });
    });
  });
});
```

### 👉 2.3 Promises: Inversion of Control Ka Solution

Promise ek JavaScript Object hai jo future me complete hone wale async task ka **Trustable Contract** hai. Iske 3 states hote hain:

- **1. Pending:** Initial state (kaam abhi chal raha hai).
- **2. Fulfilled (Resolved):** Kaam successfully ho gaya (triggers `.then()` callback).
- **3. Rejected:** Kaam fail ho gaya / error aa gaya (triggers `.catch()` callback).

```
// ✅ Clean Promise Contract
const placeFoodOrder = new Promise((resolve, reject) => {
  const isRestaurantOpen = true;
  setTimeout(() => {
    if (isRestaurantOpen) {
      resolve({ orderId: 9042, item: "Paneer Butter Masala" }); // Fulfilled!
    } else {
      reject(new Error("Restaurant is closed right now!")); // Rejected!
    }
  }, 1000);
});

// Consuming the Promise:
placeFoodOrder
  .then((order) => console.log("Order Delivered 🍽️:", order))
  .catch((err) => console.error("Order Failed ❌:", err.message));
```



### *Indian Analogy: Haldiram Food Token vs Live Order*

- **Synchronous (Blocking):** Aap counter pe khade ho, jab tak Chhole Bhature ban nahi jate, cash counter kisi aur customer se order nahi lega (Puri line jam!).
- **Promise (Non-Blocking):** Counter ne aapko ek **Token (Promise)** de diya aur bola "Pending". Aap aage badh gaye. Jab food ready hoga (Fulfilled), screen pe number flash hoga aur aap `.then()` me jakar khana utha loge!

#### ⚡ 2.4 Async / Await: Modern Clean Standard

`async/await` Promises ke upar likha gaya syntactic sugar hai jo `async` code ko clean synchronous jaisa dikhata hai with simple `try/catch` error handling:

snippet\_async\_await\_clean.js

Async/Await with try/catch

```
async function checkoutUserCart(userId) {
  try {
    const user = await fetchUser(userId);
    const cart = await fetchUserCart(user.id);
    const receipt = await processPayment(cart.total);
    console.log("Payment Success! Receipt ID:", receipt.id);
  } catch (err) {
    console.error("Checkout failed gracefully:", err.message);
  }
}
```



# Visual Code Anatomy: Async / Await & Promise Execution Flow

SYNTAX &  
KEYWORD  
BREAKDOWN

```
async function loadUserData(userId) {  
  try {  
    const user = await fetchUserFromDB(userId);  
    return user;  
  } catch (err) {  
    console.error("Failed:", err.message);  
  }  
}
```



## 1. async Keyword

Function ko promise producer banata hai – always returns a Promise auto-wrapped!



## 2. await Keyword

**Pause Point:** Engine ko bolta hai 'DB se data aane tak ruk ja, par main JS thread freeze mat kar!'



## 3. Promise Offload

Background task jo resolve hone par unwrap ho jata hai.



## 4. try / catch Net

**Safety Net:** Kisi bhi error ya rejected promise ko turant pakad leta hai.

### 3. Destructuring – Object & Array

#### 3.1 Problem Kya Thi?

Pehle hum object ki values access karne ke liye baar baar `user.name`, `user.age`, `user.city` likhte the. 10 properties hongi toh 10 lines waste. Agar spelling galat ho toh `undefined` silently aata tha – koi error nahi!

3\_object\_destructuring.js

Rename + Defaults + Nested

```
// ❌ OLD WAY – repetitive and error prone
const user = { name: 'Priya', age: 24, city: 'Pune', role: 'Dev' };
const name1 = user.name; const age1 = user.age; // tedious x 4

// ✅ Destructuring – one line, zero repetition
const { name, age, city } = user;

// Rename while destructuring: userId → id
const apiResponse = { userId: 101, userName: 'aarav_dev', status: 'active' };
const { userId: id, userName: username, role = 'viewer' } = apiResponse;
//           ↑ rename   ↑ rename   ↑ default value if key missing!
console.log(id, username, role); // 101 'aarav_dev' 'viewer'

// Nested Object Destructuring
const order = { id: 501, customer: { name: 'Rahul', city: 'Delhi' }, amount: 1499 };
const { customer: { name: custName }, amount } = order;
console.log(custName, amount); // 'Rahul' 1499

// Array Destructuring – position based
const scores = [98, 75, 62, 89];
const [first, second, , fourth] = scores; // skip 3rd with empty comma

// ★ Classic Interview: Swap without temp variable!
let a = 10, b = 20;
[a, b] = [b, a]; // a = 20, b = 10 ✅

// Real Express.js use – ye destructuring hi hai!
app.get('/users/:id', (req, res) => {
  const { id } = req.params; // Object destructuring
  const { page = 1 } = req.query; // Destructuring with default!
});
```



## Code Anatomy: Object Destructuring with Rename & Default

LINE BY  
LINE

```
const { userId: id, role = 'viewer' } = apiResponse;
```

- | **userId** — Original key in the object to look up
- | **id** — New local variable name (renamed)
- | **role = 'viewer'** — Default if key is missing
- | **apiResponse** — Object being destructured



### Interview Gotcha

"Variable swap without  
temp variable?" → [a,  
b] = [b, a] –  
Array destructuring!



### Used Everywhere

```
const { id } =  
req.params – ye  
har Express route mein  
use hota hai.  
Destructuring hai ye!
```

## 4. Spread (...) & Rest (...) – Same Symbol, Opposite Job

### 4.1 Problem Kya Tha?

Object copy/merge ke liye `Object.assign()` aur Array merge ke liye `concat()` likhna padta tha. Aur functions mein variable arguments handle karna messy tha. . . . ne dono problems ek hi symbol se solve kar diye.

### 4.2 Spread – iterable ko "phailao"

Jab . . . value ke **right side** pe assignment mein use ho – ye Spread hai. Object/array ko uske parts mein tod deta hai.

4\_spread\_rest.js

Spread + Rest Patterns

```
// ===== SPREAD =====

// 1. Immutable Object Update (Most used in Node.js + React)
const user = { name: 'Amit', age: 25, city: 'Mumbai' };
const updated = { ...user, age: 26, designation: 'Lead' };
// user.age is still 25 – original NEVER mutated ✓

// 2. Merge two objects (later key wins)
const defaults = { theme: 'dark', lang: 'en' };
const userPrefs = { lang: 'hi' }; // override lang
const config = { ...defaults, ...userPrefs }; // { theme:'dark', lang:'hi' }

// 3. Array merge + clone
const arr1 = [1, 2], arr2 = [3, 4];
const merged = [...arr1, ...arr2]; // [1, 2, 3, 4]

// 4. Spread into function args
const nums = [4, 18, 7];
console.log(Math.max(...nums)); // 18

// ===== REST =====

// 1. Rest in function – collect remaining args into array
function calcTotal(gst, ...prices) {
  const sub = prices.reduce((s, p) => s + p, 0);
  return sub + (sub * gst / 100);
}
calcTotal(18, 500, 300, 200); // (1000) + 18% = 1180

// 2. Rest in destructuring – extract specific, keep rest
const emp = { name: 'Kiran', id: 'E204', salary: 95000, dept: 'Eng' };
const { salary, ...publicProfile } = emp;
// salary is private. publicProfile = { name, id, dept } – salary gone! ✓
```



## Anatomy: Spread vs Rest – Same Symbol, Different Context

KEY  
DIFFERENCE

**SPREAD** — Right side of = — **Expands** one into many

`const copy = { ...obj }; — Opens the object up`

**REST** — Left side of = — **Collects** many into one

`const { x, ...rest } = obj; — Gathers remaining`

`function foo(...args) {} — All arguments → array`



### Interview Q

"Spread aur Rest mein kya fark hai?" — Same ... symbol, but **context** decide karta hai kya karega!



### Rule

Rest parameter hamesha function ki **last position** pe hona chahiye!  
`foo(...a, b) = SyntaxError!`

## 5. Arrow Functions – Short Syntax + Fixed 'this'

### 5.1 Problem: Regular Functions Ka 'this' Dhoka Deta Tha

JavaScript mein `this` ka value decide hota hai **function kaise call hua** – not where it was written. Regular functions mein `this` runtime pe change ho jaata tha, especially callbacks mein class ka context kho jaata tha.

5\_arrow\_functions.js

this Binding + Syntax

```
// ❌ Regular Function in setTimeout – 'this' gets lost!
function Timer() {
  this.seconds = 0;
  setInterval(function() {
    this.seconds++; // ⚡ 'this' is NOT Timer here – it's undefined/Window!
  }, 1000);
}

// ✅ Arrow Function – LEXICAL 'this' (locked to definition scope)
function TimerFixed() {
  this.seconds = 0;
  setInterval(() => {
    this.seconds++; // ✅ 'this' correctly refers to TimerFixed!
  }, 1000);
}

// Arrow Syntax – from verbose to concise
const add = function(a, b) { return a + b; }; // regular
const add = (a, b) => { return a + b; };      // arrow full
const add = (a, b) => a + b;                  // implicit return ✅
const double = n => n * 2;                    // single param, no ()
const getUser = id => ({ id, createdAt: new Date() }); // return object: wrap in ()

// ⚠️ WHEN NOT TO USE Arrow Functions
const counter = {
  count: 0,
  // ❌ Arrow – 'this' is NOT counter, it's outer scope!
  increment: () => { this.count++; },
  // ✅ Regular method – 'this' correctly refers to counter
  decrement() { this.count--; }
};
```





# Anatomy: Arrow vs Regular – this Binding

INTERVIEW  
CRITICAL

**Regular Function** — **Dynamic this** — Decided at CALL TIME

obj.method() → this = obj

setTimeout(fn) → this = undefined (strict mode)

**Arrow Function** — **Lexical this** — Decided at DEFINITION TIME

Inherits this from surrounding scope – LOCKED forever

## Interview Q

"Arrow aur regular function mein kya fark hai?" → this binding! Arrow = lexical, Regular = dynamic.

## No arguments + No new

Arrow ke paas apna arguments object nahi hota. Constructor bhi nahi bana sakte (new Arrow() → Error).

## 6. Template Literals – Backtick Strings

### 6.1 Problem: String Concatenation Hell

6\_template\_literals.js

Old Way vs Template Literal

```
const name = 'Anjali', score = 94, rank = 2;

// ❌ OLD – ugly, breaks easily, can't do multiline
const msg = "Hello " + name + "! Score: " + score + ", Rank: #" + rank;

// ✅ Template Literal – clean, any expression inside ${}
const msg = `Hello ${name}! Score: ${score}, Rank: #${rank}`;

// Expressions inside ${} – any valid JS!
const bill = `Total with GST: ₹${(score * 10 * 1.18).toFixed(2)}`;

// Multi-line (no \n needed!)
const email = `Dear ${name},
Your interview is confirmed.
Report at 10 AM sharp.`;

// Backend use – Log messages (most common!)
console.log(`[${new Date().toISOString()}] Server started on port 3000`);
console.log(`User ${userId} requested ${req.method} ${req.url}`);
```

## 7. Closures – Function Remembers Its Birthplace

### 📖 7.1 Closure Kya Hai?

Jab ek **inner function** apne **outer function** ke **variables** ko tab bhi access kar sakta hai jab outer function ka execution khatam ho chuka ho – ye **Closure** hai. Inner function apne surrounding lexical scope ka reference memory mein rakhta hai.

7\_closures.js

Private State + Real Backend Use

```
// makeCounter run hoke khatam ho jata hai...
function makeCounter() {
  let count = 0; // Private – bahar se directly access nahi kar sakte!

  return {
    increment() { count++; console.log(`Count: ${count}`); },
    getCount() { return count; }
  };
}

const counter = makeCounter(); // khatam hua... lekin count yaad hai!
counter.increment(); // Count: 1
counter.increment(); // Count: 2
console.log(counter.count); // undefined – PRIVATE! ✅

// Real Node.js use: Middleware Factory (closures in Express!)
function requireRole(role) { // outer function
  return function(req, res, next) { // inner function (middleware)
    if (req.user?.role !== role) { // 'role' from closure!
      return res.status(403).json({ error: 'Access Denied' });
    }
    next();
  };
}

// Usage:
app.delete('/admin/user', requireRole('admin'), deleteUserHandler);
// requireRole('admin') returns a middleware that "remembers" 'admin' via closure!
```



# Anatomy: Closure Memory

## Lifecycle

INTERVIEW  
CRITICAL

**makeCounter()** called — Stack frame created with **count = 0**

Returns inner object → stored in heap as **counter**  
makeCounter stack frame destroyed × (done)

**BUT:** inner functions hold **[[Environment]]** reference to outer scope  
└ **count** survives in heap as long as **counter** object exists!



### Interview Q

"Closure kya hai?" → Function + uska surrounding lexical environment. Data encapsulation ke liye use hota hai.



### Used Where?

Middleware factories, Rate limiters, Memoization, Module pattern – real Node.js code mein closures hi hain!

## 8. map / filter / reduce – Array Ki Power Trio

### 8.1 Problem: for loops se data transform karna messy tha

Array transform karne ke liye hum for loop likhte, nayi array manually push karte – verbose aur error prone. ES6 ne 3 declarative alternatives diye jo shorter, readable, aur immutable hain.

#### map() – Har Element Transform Karo (Same Size)

8\_map\_filter\_reduce.js

Array Power Trio

```
const employees = [
  { name: 'Sneha', salary: 60000, dept: 'HR' },
  { name: 'Ravi', salary: 85000, dept: 'Eng' },
  { name: 'Ankit', salary: 72000, dept: 'Eng' }
];

// ===== MAP – Transform every element, returns SAME SIZE array =====
const withHike = employees.map(emp => ({ ...emp, salary: emp.salary * 1.1 }));
// Original employees array UNCHANGED ✓

// ===== FILTER – Keep matching elements, returns SMALLER/EQUAL array =====
const engTeam = employees.filter(emp => emp.dept === 'Eng');
// [{ name: 'Ravi'... }, { name: 'Ankit'... }]

// ===== REDUCE – Fold everything into ONE value =====
const totalPayroll = employees.reduce((acc, emp) => acc + emp.salary, 0);
// 60000 + 85000 + 72000 = 217000

// POWER MOVE: Chain all three!
const engPayrollAfterHike = employees
  .filter(emp => emp.dept === 'Eng')
  .map(emp => ({ ...emp, salary: emp.salary * 1.1 }))
  .reduce((acc, emp) => acc + emp.salary, 0);
console.log(`Eng Payroll after hike: ₹${engPayrollAfterHike}`); // ₹173700

// reduce to build an object (group by)
const byDept = employees.reduce((acc, emp) => {
  acc[emp.dept] = acc[emp.dept] || [];
  acc[emp.dept].push(emp.name);
  return acc;
}, {});
console.log(byDept); // { HR: ['Sneha'], Eng: ['Ravi', 'Ankit'] }
```



## map vs filter vs reduce – Quick Comparison

MUST  
REMEMBER

**map(fn)** — Input[n] → Output[n] — **SAME SIZE**, each element transformed  
**filter(fn)** — Input[n] → Output[≤n] — **SMALLER**, only matching kept  
**reduce(fn, v)** — Input[n] → Output[1] — **ONE VALUE** – number, string, or object



### Interview Q

"map aur  
forEach  
mein  
fark?"  
→ map  
returns  
a NEW  
array.  
forEach  
returns  
undefined  
– sirf  
side  
effects  
ke liye!



### Chaining

`arr.filter(...).map(...).reduce(...)`  
– real production code yahi hai! Teen method chain mein  
powerful transformations.

## *Node.js Architecture & Event Loop Internals*

---

Topics Explanation with Indian Examples

## 1. Single-Threaded Event Loop & The Blocking Problem

### 🤔 1.1 Problem Kya Thi? (Apache Multi-Thread vs Node.js Event Loop)

Traditional web servers (jaise Apache PHP ya Java Tomcat) har incoming user request ke liye ek naya OS Thread banate the. Agar 10,000 concurrent users aate the, toh 10,000 threads create hote the jisse server ki RAM khatam ho kar server crash ho jata tha (The C10K Problem).

**Node.js Ka Solution:** Node.js ne **Single Main Thread + Non-Blocking I/O Event Loop** model apnaya. Ek hi thread millions of requests handle karta hai by offloading heavy work!

snippet\_blocking\_vs\_nonblocking.js

Blocking vs Non-Blocking Demo

```
const fs = require('fs');

// ⚠️ BLOCKING (DANGEROUS IN PRODUCTION):
// Pura Event Loop freeze ho jayega jab tak file read complete na ho!
const dataSync = fs.readFileSync('./heavy_report.pdf');
console.log("Sync Read Complete - Baaki users ka request atak gaya tha!");

// ✅ NON-BLOCKING (PRODUCTION STANDARD):
// Task Libuv background thread ko diya, JS aage badh gaya bina ruke:
fs.readFile('./heavy_report.pdf', (err, data) => {
  if (err) return console.error(err);
  console.log("Async Read Complete - Zero blocking for other users! 🚀");
});
console.log("🏁 Main Thread is free to serve other 10,000 users!");
```

### ⚙️ 1.2 V8 Engine & Libuv C++ Threadpool

- **Chrome V8 Engine:** Google dwara banaya gaya C++ engine jo JavaScript ko JIT (Just-In-Time) compile karke direct fast Machine Code me convert karta hai (Ignition Interpreter + TurboFan Compiler).
- **Libuv Library:** C-language library jo operating system ke low-level non-blocking APIs (Linux epoll, macOS kqueue, Windows IOCP) se baat karti hai.
- **Libuv Threadpool:** Default me 4 Background Worker Threads hote hain jo 4



heavy operations handle karte hain: File System (fs), Crypto (Hashing), DNS Lookups, aur Zlib (Compression).

snippet\_threadpool\_crypto.js

Libuv 4-Thread Parallel Hashing

```
const crypto = require('crypto');

// Setting threadpool size (Default is 4, max is 1024):
process.env.UV_THREADPOOL_SIZE = 4;

const start = Date.now();
for (let i = 1; i <= 4; i++) {
  crypto.pbkdf2('secret_password', 'salt', 100000, 512, 'sha512', () => {
    console.log(`Thread #${i} Done in: ${Date.now() - start}ms`);
  });
}
console.log("All 4 hashing tasks offloaded in parallel to Libuv Threadpool!");
```



### *Indian Real-Life Analogy: Haldiram Cash Counter & Kitchen Staff*

- **Main JS Thread (Cashier):** Ek akela fast cashier jo har customer ka order leta hai, token deta hai, aur turant next customer ko attend karta hai (Non-blocking).
- **Libuv Threadpool (4 Kitchen Cook):** Background me 4 cooks jo Chole Bhature talne (Heavy I/O & Crypto) ka kaam karte hain. Bhature ready hote hi bell bajti hai aur customer ko token par deliver ho jata hai!



# Visual Architecture Blueprint: Event Loop 6 Phases & Microtasks

LIBUV  
EXECUTION  
CYCLE

[ Synchronous JS Code ] → **Call Stack** (Executes Line-by-Line)  
| (When Stack is Empty)  
▼  
◀ **VIP Microtask Queue** → **process.nextTick()** & **Promise.then()** (Runs IMMEDIATELY between each phase!)  
▼  
**Phase 1: TIMERS** → **setTimeout()** & **setInterval()** callbacks  
**Phase 2: PENDING** → Low-level I/O callbacks (TCP stream errors, UDP)  
**Phase 3: IDLE / PREPARE** → Internal Libuv engine bookkeeping  
**Phase 4: POLL** → New incoming I/O events (fs read, incoming HTTP requests)  
**Phase 5: CHECK** → **setImmediate()** callbacks (Guaranteed after I/O!)  
**Phase 6: CLOSE** → Cleanups like **socket.on('close')**  
♻️ Cycle repeats smoothly while active timers/sockets exist!



**process.nextTick()**

Event Loop ke kisi bhi phase ka wait nahi karta. Current operation khatam hote hi Call Stack ke turant baad execute hota hai (Microtask VIP pass).



**setImmediate()**

Hamesha **Poll Phase** (I/O) ke turant baad **Check Phase** me execute hota hai. I/O callbacks ke andar **setImmediate()** hamesha **setTimeout()** se pehle chalta hai!

## 🏁 1.3 Event Loop Race: nextTick vs setImmediate vs setTimeout

```
console.log("1. Synchronous Main Line");

setTimeout(() => console.log("4. Timers Phase: setTimeout (0ms)"), 0);
setImmediate(() => console.log("5. Check Phase: setImmediate"));

Promise.resolve().then(() => console.log("3. Microtask: Promise.then"));
process.nextTick(() => console.log("2. VIP Microtask: process.nextTick"));

console.log("1. Synchronous End Line");

/* Output Order:
  1. Synchronous Main Line
  1. Synchronous End Line
  2. VIP Microtask: process.nextTick (Runs before any phase!)
  3. Microtask: Promise.then (Runs in microtask queue)
  4. Timers Phase: setTimeout (0ms) (Timers phase)
  5. Check Phase: setImmediate (Check phase)
*/
```

## 2. Complete Architecture Working Demo

chapter02\_master\_demo.js

Node.js Architecture Demo

```
const fs = require('fs');
const crypto = require('crypto');

console.log("--- Node.js Event Loop Architecture Demo ---");

// 1. Microtask VIP Queue:
process.nextTick(() => {
  console.log("[VIP Microtask] process.nextTick executed before Event Loop phases"
});

// 2. I/O Poll Phase & Check Phase Execution:
fs.readFile(__filename, () => {
  console.log("[I/O Phase] File read callback completed");

  // Inside I/O callback, setImmediate is GUARANTEED to run before setTimeout!
  setTimeout(() => console.log("[Timers Phase inside I/O] setTimeout (0ms)", 0));
  setImmediate(() => console.log("[Check Phase inside I/O] setImmediate executed I
});

console.log("[Call Stack] Main synchronous script completed!");
```

| Component        | Role in Architecture                   | Execution Timing                         |
|------------------|----------------------------------------|------------------------------------------|
| Call Stack       | Synchronous JavaScript code execution  | Line-by-line first priority              |
| process.nextTick | Highest priority microtask queue       | Runs immediately after current operation |
| Promise Queue    | Microtask queue for async promises     | Runs after nextTick, before next phase   |
| Timers Phase     | Executes setTimeout & setInterval      | First phase of Event Loop                |
| Poll Phase       | Retrieves I/O events & reads data      | Fourth phase of Event Loop               |
| Check Phase      | Executes setImmediate callbacks        | Immediately follows Poll phase           |
| Libuv Threadpool | 4 OS threads for FS, Crypto, DNS, Zlib | Background C++ workers                   |

### 3. require() – Node.js Module System

#### 3.1 Problem Kya Tha? (Why Modules Exist)

Agar sab code ek hi file mein hota (jaise pehle tha), toh ek badi project mein 10,000 lines ki single file ho jaati. Name conflicts, maintenance nightmare, testing impossible. Solution: **Modules** – har file apna independent scope rakhti hai.

#### 3.2 require() Andar Se Kya Karta Hai? (5 Steps)

3\_require\_module\_system.js

CommonJS Module Mechanics

```
// ===== FILE: mathUtils.js (ek module) =====
const PI = 3.14159;

function circleArea(r) { return PI * r * r; }
function rectArea(l, w) { return l * w; }

// module.exports – ye hi bahar jaata hai!
module.exports = { circleArea, rectArea };
// PI bahar nahi jaata – private raha! ✓

// ===== FILE: app.js =====
const { circleArea, rectArea } = require('./mathUtils'); // .js optional
console.log(circleArea(5)); // 78.53...

// Built-in modules (no path needed)
const fs = require('fs'); // Node built-in
const path = require('path'); // Node built-in
const crypto = require('crypto'); // Node built-in

// Third-party (from npm, installed in node_modules)
const express = require('express'); // npm install express se aaya

// KEY FEATURE: Module Caching!
const mod1 = require('./mathUtils');
const mod2 = require('./mathUtils');
console.log(mod1 === mod2); // true – same object! File sirf ek baar exe
```



# Anatomy: require() ke 5 Internal Steps

UNDER THE HOOD

**Step 1: RESOLVE** — './mathUtils' → full absolute path find karo

**Step 2: CACHE CHECK** — Kya ye file pehle load ho chuki hai? Agar haan → SAME object return!

**Step 3: LOAD** — File read karo (synchronously – ye ek baar hi hota hai startup pe)

**Step 4: WRAP** — Code ko function mein wrap: (function(exports, require, module, \_\_filename, \_\_dirname) { ... })

**Step 5: EVALUATE + CACHE** — Execute karo, module.exports cache mein store karo



## Interview Q

"require() synchronous hai ya async?" → Synchronous! Startup pe files load hoti hain. Request handling ke beech nahi.



## Caching

Module caching ki wajah se same module 100 files mein require karo – file sirf ek baar execute hogi. Memory efficient!

## 3.3 module.exports vs exports – Confusion Clear Karo

### module\_exports\_vs\_exports.js

### The Difference

```
// exports = shortcut reference to module.exports
// Initially: exports === module.exports (same object)

// ✅ SAFE: Adding properties (both work)
exports.greet = function(name) { return `Hello ${name}`; };
module.exports.bye = function(name) { return `Bye ${name}`; };

// ❌ DANGER: Reassigning exports breaks the reference!
exports = { newFn: () => {} }; // exports is now a NEW object – module.e

// ✅ CORRECT: Reassign module.exports directly
module.exports = { newFn: () => {} }; // This is the actual exported val

// Best practice: Always use module.exports for clarity!
```

## 4. package.json + npm – Project ka DNA

### 4.1 Problem Kya Tha?

Bina package.json ke kisi ko project share karna = 100MB  
node\_modules folder bhi bhejni padti. Aur ye bhi clear nahi  
hota ki project ko chalaane ke liye kya kya chahiye.

package.json ek **manifest file** hai – project ki poori  
identity is mein hoti hai.

#### package.json

#### Project Manifest – Every Field Explained

```
{
  "name": "my-express-api",
  "version": "1.0.0",
  "description": "Job Board REST API",
  "main": "app.js",           // Entry point – node app.js se start hoga

  "scripts": {
    "start": "node app.js",    // npm start
    "dev": "nodemon app.js",   // npm run dev (auto-restart on cha
    "test": "jest"             // npm test
  },

  "dependencies": {
    "express": "^4.18.2",      // Production dependency – npm install se a
    "dotenv": "^16.0.3"        // Environment variables
  },

  "devDependencies": {
    "nodemon": "^3.0.0"        // Only in development – NOT in production!
  }
}

// npm commands – must know!
// npm init -y                → package.json banao (defaults)
// npm install express         → install + dependencies mein add
// npm install nodemon --save-dev → devDependencies mein add
// npm install                 → package.json dekh ke sab install karo
// npm uninstall express       → remove
```



## Anatomy: Version Notation

COMMON  
CONFUSION

(^ ~ \*)

"express": "^4.18.2" — **Caret ^** — 4.x.x allowed (major version fixed)

"express": "~4.18.2" — **Tilde ~** — 4.18.x allowed (minor version fixed)

"express": "4.18.2" — **Exact** — Sirf ye exact version



### .gitignore

Hamesha  
node\_modules/  
ko .gitignore  
mein daalo! Sirf  
package.json  
push karo — npm  
install se sab  
wapas aayega.



### Interview Q

"dependencies aur  
devDependencies  
mein kya fark hai?"  
→ dev = sirf  
development (tools).  
production build  
mein nahi jaata!



## *File System & Streams Mastery*

---

Topics Explanation with Indian Examples

## 1. fs Module: Sync vs Async vs Promises

### 🤔 1.1 Problem Kya Thi? (Why fs.readFileSync Crashes Production)

Jab aap `fs.readFileSync()` use karte ho, toh jab tak disk se 500 MB file read nahi hoti, JavaScript ka akela main thread freeze ho jata hai. Is doran baki 5,000 users ki requests line me atki rehti hain aur site slow/down ho jati hai.

**Solution:** Hamesha Non-blocking `fs.promises` ya **Streams** use karein!

snippet\_fs\_modern.js

Modern fs.promises Example

```
const fs = require('fs').promises;

async function manageFiles() {
  try {
    // 1. Write file asynchronously:
    await fs.writeFile('./app_log.txt', 'User logged in at 10:00 AM\n');

    // 2. Append without overwriting:
    await fs.appendFile('./app_log.txt', 'User uploaded invoice #108\n');

    // 3. Read file cleanly:
    const content = await fs.readFile('./app_log.txt', 'utf-8');
    console.log("File Content Loaded:\n", content);
  } catch (err) {
    console.error("File Operation Failed:", err.message);
  }
}

manageFiles();
```

### 📦 1.2 Buffers: What is Binary Data in Node.js?

- **Buffer:** V8 Heap memory ke bahar allocate kiya gaya raw binary memory chunk hai. JavaScript me pehle binary data handle karne ka tarika nahi tha, isliye Node.js ne `Buffer` class banayi.
- Data memory me **Octets (Bytes in Hexadecimal format 00-FF)** ke roop me store hota hai.

```
// 1. Create buffer from String:
const buf = Buffer.from('NodeJS');
console.log(buf); // <Buffer 4e 6f 64 65 4a 53> (Hex values of ASCII characters)
console.log(buf.toString('utf-8')); // "NodeJS"

// 2. Allocate fixed safe zero-filled memory:
const emptyBuf = Buffer.alloc(5); // <Buffer 00 00 00 00 00> (5 bytes reserved)
```

### 1.3 Streams & Backpressure Management

Jab aapko 5 GB ki video ya 1 crore database records transfer karne hote hain, toh pura file RAM me load karne par JavaScript heap out of memory error aayega aur server crash ho jayega.

- **Streams:** Data ko **64 KB ke chhote Chunks** me tod kar ek jagah se dusri jagah pipe karte hain.

- **4 Types of Streams:**

1. **Readable Stream:** Jahan se data padha jata hai (e.g.

`fs.createReadStream`, HTTP req).

2. **Writable Stream:** Jahan data likha jata hai (e.g.

`fs.createWriteStream`, HTTP res).

3. **Duplex Stream:** Jo padh aur likh dono sake (e.g. TCP Net Socket).

4. **Transform Stream:** Jo data aane par usse modify/compress kare (e.g. `zlib.createGzip()`).

- **Backpressure:** Jab reader tezi se chunks bhej raha ho aur writer slow ho, toh internal buffer bhar jata hai. `.pipe()` ya `stream.pipeline()` reader ko automatically pause kar deta hai aur writer ke drain hone par resume karta hai!



## Indian Real-Life Analogy: Chhat Ki 1000L Tanki (Buffer) & Pipe (Stream)

- **readFile (Crash):** 1000 Liter pani ek sath room me daal diya (Flood & Furniture ruined).
- **Streams & Pipe:** Chhat ki tanki se 1/2 inch ke pipe ke zariye glass-by-glass pani lena — sirf 1 glass jitna RAM use hota hai chahe tanki 10,000 Liter ki kyu na ho!



## Visual Architecture Blueprint: Stream Pipeline & Backpressure Flow

FLOW CONTROL  
MECHANICS

```
[ 10 GB Video File ] → fs.createReadStream() (Reads 64 KB Chunks)
|
▼ Chunks emitted via 'data' events: [ Chunk 1 (64KB) ] → [ Chunk 2 (64KB) ]
|
. pipe() Flow Control (Monitors internal highWaterMark buffer = 16KB)
|
▼
zlib.createGzip() → [ Transform Stream compresses chunks on-the-fly ]
|
(If Disk write is slow: Readable is PAUSED automatically!)
▼ (Resume when 'drain' event fires)
[ Compressed File ] ← fs.createWriteStream('video.mp4.gz')
```



### highWaterMark

Stream buffer ki maximum capacity (Default: 64 KB for files, 16 KB for HTTP). Buffer bharne par write() false return karta hai.



### stream.pipeline()

.pipe() me error handling manual karni padti thi. pipeline() auto-cleanup karta hai aur stream leak nahi hone deta!

## 2. Complete Streams Production Pipeline Demo

chapter03\_master\_demo.js

Safe Streaming with pipeline

```
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream');

console.log("--- Memory Safe Stream Compression Pipeline ---");

const sourceFile = './app_log.txt';
const destinationFile = './app_log.txt.gz';

// Setup Readable, Transform, and Writable streams:
const readStream = fs.createReadStream(sourceFile);
const gzipStream = zlib.createGzip();
const writeStream = fs.createWriteStream(destinationFile);

// Use pipeline() for automatic error handling and memory cleanup:
pipeline(readStream, gzipStream, writeStream, (err) => {
  if (err) {
    console.error("❌ Stream Pipeline Crashed:", err.message);
  } else {
    console.log("✅ File Compressed Successfully with < 20MB RAM usage!");
  }
});
```

| Stream Feature                 | How It Works                                               | Why Important                                       |
|--------------------------------|------------------------------------------------------------|-----------------------------------------------------|
| <code>createReadStream</code>  | Disk se 64 KB ke tukdo me data emit karta hai              | RAM kabhi overflow nahi hoti                        |
| <code>createWriteStream</code> | Data chunks ko destination file/socket par write karta hai | Safe persistent storage                             |
| <code>zlib.createGzip</code>   | Chunks ko receive karke on-the-fly compress karta hai      | Zero disk intermediary required                     |
| <code>pipeline()</code>        | Multiple streams ko jorta hai aur auto-close karta hai     | Memory leak aur dangling sockets se bachaata hai    |
| Backpressure                   | Flow-control mechanism jo reader ko pause/resume karta hai | Fast reader slow writer ko overpower nahi kar sakta |

## *Raw Node.js HTTP Server*

---

Topics Explanation with Indian Examples

## 1. `http.createServer` & TCP Stream Chunks

### 🤔 1.1 Problem Kya Thi? (How Browsers & TCP Sockets Communicate)

Browser jab kisi server par HTTP request bhejta hai, toh wo ek hi jhatke me pura data nahi deta — wo network ke zariye **TCP Packets (Chunks)** me data transmit karta hai. Agar server par aap POST body ko stream chunks me receive karke buffer nahi karoge, toh payload gayab ho jayega!

- req is a **Readable Stream**: Request payload 'data' events ke form me chunks me flow hota hai.
- res is a **Writable Stream**: Response headers `res.writeHead()` se aur body `res.end()` se TCP connection close karke bhejte hain (Bina `res.end()` browser hamesha ghumta rahega / infinite loading!).

snippet\_raw\_http\_post.js

Manual TCP Chunk Buffering

```
const http = require('http');

const server = http.createServer((req, res) => {
  const { method, url } = req;

  if (method === 'POST' && url === '/api/order') {
    const bodyChunks = [];

    // 1. Listen to incoming binary chunks:
    req.on('data', (chunk) => {
      bodyChunks.push(chunk);
    });

    // 2. When all TCP packets finish, assemble & parse:
    req.on('end', () => {
      const rawBody = Buffer.concat(bodyChunks).toString('utf-8');
      const parsedData = JSON.parse(rawBody);

      // 3. Send HTTP Status 201 + JSON Response:
      res.writeHead(201, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ status: 'Success', orderId: 108, item: parsedData }));
    });
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end("Route Not Found in Raw HTTP Server");
  }
});

server.listen(3000, () => console.log("Raw HTTP Server running on http://localhost:3000"));
```



## Indian Real-Life Analogy: Postman & Parcel Assembly

- **TCP Chunks (req.on('data')):** Postman ek bada bed 3 alag-alag packets me roz laakar deta hai.
- **Buffer.concat (req.on('end')):** Jab teeno packets aa jaate hain, aap unhe jodh kar pura bed tayyar karte hain aur use accept karte hain!



## Visual Architecture Blueprint: Raw HTTP Socket Flow & Buffering

TCP SOCKET  
LIFECYCLE

```
Client Browser → [ HTTP POST /api/order ] → TCP Socket Stream
```

|

▼ req.on('data', chunk ⇒ bodyChunks.push(chunk)) [ Ingests 16KB Network Packets ]

|

▼ req.on('end', () ⇒ ...) [ Signal: Stream Closed by Client ]

Buffer.concat(chunks).toString() → JSON.parse(rawBody)

▼

res.writeHead(201, headers) → res.end(payload) → Connection Closed ✓

⚠ res.end() Missing

📄 Content-Type



## 2. Complete Raw HTTP REST Server Demo

chapter04\_master\_demo.js

Production Raw HTTP Server

```
const http = require('http');

const server = http.createServer((req, res) => {
  const { method, url } = req;

  // GET /api/health
  if (method === 'GET' && url === '/api/health') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify({ status: 'Healthy', uptime: process.uptime() }));
  }

  // POST /api/echo
  if (method === 'POST' && url === '/api/echo') {
    let chunks = [];
    req.on('data', c => chunks.push(c));
    req.on('end', () => {
      const body = Buffer.concat(chunks).toString();
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ received: JSON.parse(body) }));
    });
    return;
  }

  res.writeHead(404, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ error: 'Route Not Found' }));
});

server.listen(3000);
```

| Raw HTTP Step                | Code Trigger                   | Why Necessary                                                 |
|------------------------------|--------------------------------|---------------------------------------------------------------|
| <code>req.on('data')</code>  | Incoming TCP packet event      | Chunks ko array me collect karne ke liye                      |
| <code>req.on('end')</code>   | Stream termination event       | Pura payload ready hone par processing start karne ke liye    |
| <code>Buffer.concat()</code> | Binary memory stitcher         | All chunks ko ek complete raw buffer string banane ke liye    |
| <code>res.writeHead()</code> | HTTP status & response headers | Client browser ko response protocol batane ke liye            |
| <code>res.end()</code>       | Closes HTTP connection         | TCP handshake terminate karke response complete karne ke liye |

### 3. HTTP Methods – Kya Fark Hai GET / POST / PUT / PATCH / DELETE Mein?

🔴 3.1 Problem: Sirf ek method hota toh server kaise jaanta kya karna hai?

HTTP methods ek **shared convention** hain — client server ko bata raha hai ki kya operation karna hai. Method sirf ek "intent label" hai; server ultimately decide karta hai kya karega. Lekin is convention ko follow karna **REST API design** ka core hai.

#### http\_methods.js GET vs POST vs PUT vs PATCH vs DELETE

```
// GET – Data FETCH karo (READ only, No body, Cacheable)
// URL: GET /api/users → Sab users lao
// URL: GET /api/users/101 → Sirf user 101 lao

// POST – Naya resource CREATE karo (Body mein data, Not cacheable)
// URL: POST /api/users → Naya user banao (body mein name, email, role)

// PUT – Pura resource REPLACE karo (Complete replacement)
// URL: PUT /api/users/101 → User 101 ka COMPLETE data replace karo
// Body: { name: 'New', email: 'new@email.com', role: 'admin' }

// PATCH – Partial UPDATE karo (Sirf changed fields bhejo)
// URL: PATCH /api/users/101 → Sirf kuch fields update karo
// Body: { role: 'admin' } (Sirf role change hua, baki sab wahi)

// DELETE – Resource REMOVE karo
// URL: DELETE /api/users/101 → User 101 delete karo

// Summary in Raw Node.js:
const server = http.createServer((req, res) => {
  const { method, url } = req;

  if (method === 'GET' && url === '/api/users') /* 1 */
  if (method === 'POST' && url === '/api/users') /* 2 */
  if (method === 'PUT' && url === '/api/users/101') /* 3 */
  if (method === 'PATCH' && url === '/api/users/101') /* 4 */
  if (method === 'DELETE' && url === '/api/users/101') /* 5 */
});
```



# HTTP Methods –

## Quick Reference

INTERVIEW  
CRITICAL

**GET** — Read, No body, Cacheable, Idempotent

**POST** — Create, Has body, Not cacheable, NOT idempotent

**PUT** — Full Replace, Idempotent (same result if repeated)

**PATCH** — Partial Update, Not always idempotent

**DELETE** — Remove, Idempotent



### Interview Q

"PUT aur PATCH mein kya fark hai?" →  
PUT = complete replacement (sab fields bhejne padte hain).  
PATCH = partial update (sirf changed fields).



### Idempotent kya hai?

Ek hi request ek baar ya 100 baar karo — result same hoga. GET, PUT, DELETE = idempotent. POST = NOT idempotent (100 baar = 100 records!)

## 4. HTTP Status Codes – Server Ka Response Language

### 📖 4.1 Status Codes Kya Hote Hain?

Jab server response bhejta hai, saath ek 3-digit number bhi aata hai jo client ko batata hai: "Kya hua?" Status codes 5 categories mein hote hain based on first digit.

#### http\_status\_codes.js **Most Important Status Codes**

```
// ===== 2xx – SUCCESS =====
res.writeHead(200); // OK – Request successful, data wapas a
res.writeHead(201); // Created – Naya resource successfully
res.writeHead(204); // No Content – Delete successful, koi k

// ===== 3xx – REDIRECT =====
res.writeHead(301); // Moved Permanently – URL permanently c
res.writeHead(304); // Not Modified – Cache valid hai, doba

// ===== 4xx – CLIENT ERRORS (tumhari galti) =====
res.writeHead(400); // Bad Request – Invalid input data (mis
res.writeHead(401); // Unauthorized – Login nahi kiya (token
res.writeHead(403); // Forbidden – Login kiya hai lekin perm
res.writeHead(404); // Not Found – Route ya resource exist n
res.writeHead(409); // Conflict – Duplicate entry (email alr
res.writeHead(422); // Unprocessable Entity – Validation fai

// ===== 5xx – SERVER ERRORS (hamari galti) =====
res.writeHead(500); // Internal Server Error – Uncaught exce
res.writeHead(502); // Bad Gateway – Upstream server ne khai
res.writeHead(503); // Service Unavailable – Server overloa
```



# 401 vs 403 – The Most Confused Pair

INTERVIEW  
TRAP

**401 Unauthorized** — "Aap kaun hain? Token nahi hai ya expired hai!"

Isko authentication error kehte hain. Re-login karo.

**403 Forbidden** — "Hum jaante hain aap kaun ho – lekin ye kaam karne ki permission nahi!"

Isko authorization error kehte hain. Role check failed. Re-login se bhi nahi milega!



## Interview Q

"401 aur 403

mein kya

fark hai?"

→ 401 =

"Who are

you?" (auth

missing).

403 = "I

know you,

but NO!"

(permissions

missing).



## Best Practice

POST mein

always 201

Created

use karo,

not 200.

DELETE ke

baad 204

No

Content.

200 sirf

successful

GET/PUT ke

liye!

## *Express.js Fundamentals*

---

Topics Explanation with Indian Examples

## 1. Express vs Raw Node HTTP & Auto Content Negotiation

### 🤔 1.1 Problem Kya Thi? (Why Raw HTTP is Exhausting)

Raw Node HTTP me agar 20 routes banane ho, toh nested `if/else` ka pahaad ban jata hai. Har route me manual URL parsing, query string regex, chunk buffering, manual MIME type headers (`application/json`), aur manual `JSON.stringify` likhna padta hai.

**Express Ka Solution:** Express ne in sabhi pain-points ko **Micro-Router** aur **Content Negotiation** se 1-line me convert kar diya!

snippet\_express\_bootstrap.js

Express Routing & Helpers

```
const express = require('express');
const app = express();

// Built-in JSON Body Parser Middleware:
app.use(express.json());

// 1. res.send(): Automatic MIME Type detection (HTML vs Plain Text)
app.get('/', (req, res) => {
  res.send('<h1>Welcome to Express Server! 🚀 </h1>');
});

// 2. res.json(): Auto stringify + Content-Type: application/json header
app.get('/api/user', (req, res) => {
  res.json({ id: 101, name: 'Amit Sharma', role: 'Backend Dev' });
});

// 3. res.status(): Custom HTTP Status Codes
app.post('/api/user', (req, res) => {
  res.status(201).json({ message: 'User created successfully', data: req.body });
});

app.listen(4000, () => console.log('Express running on http://localhost:4000'));
```

## Indian Real-Life Analogy: Swiggy / Zomato vs Khud Cycle Chala Ke Hotel Jaana

- **Raw HTTP:** Khud cycle chala kar hotel jao, line me lago, cook se baat karo, cash gino, parcel pack karwao (100 lines of manual code).
- **Express.js:** Swiggy app par 1 click me order — payment, routing, delivery sab automatic handle ho jata hai (3 lines of clean code)!



## Visual Architecture Blueprint: Express Request Routing Engine

ROUTING &  
HELPERS

Incoming Request → `app.use(express.json())` (Auto parses body)

▼ Internal Route Table Lookup: [ METHOD + PATH ]

`GET /` → `res.send('<html>')` (Sets text/html)

`GET /api/user` → `res.json({ ... })` (Sets application/json + ETag)

`POST /api/user` → `res.status(201).json({ ... })` (Created)

▼ No Route Matched?

`404 Handler` → Returns "Cannot GET /unknown"

⚡ `res.send()` vs `res.json()`

🔒 ETag Ca



## 2. Complete Express Bootstrap Production Code

chapter05\_master\_demo.js

Express Bootstrap Server

```
const express = require('express');
const app = express();

app.use(express.json());

// In-memory dataset:
let products = [
  { id: 1, name: 'Wireless Mouse', price: 899 },
  { id: 2, name: 'Mechanical Keyboard', price: 3499 }
];

app.get('/products', (req, res) => {
  res.json({ count: products.length, data: products });
});

app.post('/products', (req, res) => {
  const { name, price } = req.body;
  const newProduct = { id: products.length + 1, name, price: Number(price) };
  products.push(newProduct);
  res.status(201).json({ message: 'Product Added', product: newProduct });
});

app.listen(4000, () => console.log('Express API live at http://localhost:4000'));
```

| Express Method                       | Functionality                          | Advantage Over Raw HTTP                          |
|--------------------------------------|----------------------------------------|--------------------------------------------------|
| <code>app.use(express.json())</code> | Body stream parser                     | Replaces manual chunks buffering & Buffer.concat |
| <code>app.get(path, handler)</code>  | Matches GET request on path            | Zero nested if/else logic required               |
| <code>res.json(data)</code>          | Stringifies & sets Content-Type header | Safe automated serialization                     |
| <code>res.status(code)</code>        | Chains HTTP status code                | Clean fluent API design                          |

### 3. req.params vs req.query vs req.body – The Big Three

🔴 3.1 Problem: Students ye teeno mix karte hain – interview mein fail!

Express mein request data 3 alag jagahon se aata hai. Galat jagah se data dhundhoge toh undefined milega. Ye clearly samajhna zaroori hai.

#### req\_data\_sources.js All 3 Sources of Request Data

```
// URL: GET /api/jobs/42?page=2&limit=10&sort=salary
// Body: { title: 'Backend Dev', company: 'Swiggy' } (POST)

app.get('/api/jobs/:id', (req, res) => {

  // 1. req.PARMS – URL ke dynamic segments (:id, :slug)
  // Source: /api/jobs/42 → :id = 42
  const { id } = req.params; // id = '42' (always a STRING)

  // 2. req.QUERY – URL ke ? ke baad waale filter params
  // Source: ?page=2&limit=10&sort=salary
  const { page = 1, limit = 10, sort = 'createdAt' } = req.query;
  // Destructuring with defaults – clean!

  // 3. req.BODY – Request body (POST/PUT/PATCH data)
  // ONLY available if express.json() middleware laga hai
  const { title, company } = req.body;

  res.json({ id, page: +page, limit: +limit, sort, title,
    // ↑ +page converts string to number!
  });

  // GET /api/jobs?page=2&limit=5&sort=salary&dept=Engineering
  // Search + Filter + Pagination – real production pattern!
  app.get('/api/jobs', (req, res) => {
    const { page = 1, limit = 10, sort = 'createdAt', dept } = req.query;
    let jobs = JOBS_DATA; // mock data
    if (dept) jobs = jobs.filter(j => j.dept === dept);
    const start = (page - 1) * limit;
    res.json({ page: +page, total: jobs.length, data: jobs.slice(start, start + limit) });
  });
```



## *req.params vs req.query vs req.body — Comparison*

MUST  
MEMORIZE

**req.params** — URL mein :id segment — /users/ 42 →  
req.params.id = '42'

**req.query** — URL mein ? ke baad — /users? page=2  
→ req.query.page = '2'

**req.body** — Request body (POST/PUT data) —  
express.json() middleware zaroori!

Note: params aur query values hamesha STRING type mein  
aati hain. Number chahiye toh Number(req.params.id) ya  
+req.query.page karo.



### Interview Q

"req.params  
aur  
req.query  
mein kya  
fark hai?"  
→ params  
= URL ka  
part  
(/users/42).  
query =  
filter  
(/users?  
page=2)



### Common Bug

req.body = undefined? →  
express.json() middleware lagana  
bhool gaye! Hamesha  
app.use(express.json())  
pehle lagao.

#### 4. dotenv – Environment Variables (Production Standard)

##### 🔴 4.1 Problem: Secrets ko code mein hardcode karna deadly hai!

Agar DB password, JWT secret, API keys seedha code mein likhoge aur GitHub pe push karo — koi bhi dekh sakta hai. Data breach, account hack! Solution:

**Environment Variables** — secrets .env file mein rakhte hain jo gitignore hoti hai.

.env + app.js

Environment Variables Setup

```
// ===== .env file (NEVER commit to git!) =====
PORT=3000
NODE_ENV=development
DB_URI=mongodb://localhost:27017/myapp
JWT_SECRET=my_super_secret_key_change_in_prod
API_KEY=sk-abc123xyz

// ===== .gitignore =====
node_modules/
.env

// ===== app.js =====
require('dotenv').config(); // FIRST LINE! Pehle env vars load krlo

const express = require('express');
const app = express();

// Ab process.env se access karo – koi hardcoding nahi!
const PORT = process.env.PORT || 3000;
const DB_URI = process.env.DB_URI;
const IS_PROD = process.env.NODE_ENV === 'production';

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT} [${process.env.NODE_ENV}]`);
});
```



## *Indian Analogy: Locker Room*

### *Combination*

- **Hardcoded Secret:** Apna locker combination apni exam copy ke cover pe likh dena — koi bhi dekh sakta hai!
- **.env file:** Combination alag personal notebook (gitignored) mein rakhna — code share karo, secret safe rahe!

## *Middleware Architecture & Pipeline*

---

Topics Explanation with Indian Examples

## 1. The (req, res, next) Pipeline & Error Handling

### 🤔 1.1 Problem Kya Thi? (Code Duplication Across Routes)

Agar aapke backend me 50 routes hain aur sabhi routes me check karna hai: "Kya user logged in hai?", "Request ka execution time kya tha?", "Kya incoming payload valid hai?". Agar middleware na ho, toh ye code har route ke andar 50 baar copy-paste karna padega!

**Middleware Solution:** Request aane se lekar Response bhejne ke beech me lagaye gaye functions ki chain ko **Middleware Pipeline** kehte hain.

snippet\_middleware\_pipeline.js

Middleware Chain & next()

```
const express = require('express');
const app = express();

// 1. Application-Level Logger Middleware:
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ⚡ ${req.method} ${req.url}`);
  next(); // ➡ CRITICAL: Hands over control to the next middleware!
});

// 2. Route-Specific Auth Guard Middleware:
function requireAuth(req, res, next) {
  const token = req.headers['authorization'];
  if (token === 'Bearer secret-token-123') {
    next(); // User allowed to proceed!
  } else {
    res.status(401).json({ error: 'Unauthorized: Access Token Invalid or Missing' });
  }
}

// 3. Protected Route using Auth Guard:
app.get('/dashboard', requireAuth, (req, res) => {
  res.json({ secretData: "Welcome to the VIP Admin Dashboard!" });
});

// 4. Global 4-Argument Error Handler (err, req, res, next):
app.use((err, req, res, next) => {
  console.error("Global Error Caught:", err.message);
  res.status(err.status || 500).json({ error: err.message });
});
```



## Indian Real-Life Analogy: Delhi Metro Security Check Gates

- **Gate 1 (Baggage Scanner):** Request inspect hoti hai — pass hone par bolta hai `next()`.
- **Gate 2 (Metal Detector / Auth Guard):** Agar pass ya ticket nahi hai, guard wahi se return kar deta hai (401 Unauthorized).
- **Gate 3 (Ticket Flap Gate):** Sabhi checks pass hone par hi Metro Platform (Route Controller) tak entry milti hai!



## Visual Architecture Blueprint: Express Middleware Pipeline & Error Routing

PIPELINE  
EXECUTION  
FLOW

```
HTTP Request → [Logger Middleware] → next()
|
▼
[Auth Guard Middleware] → next() (If authorized)
|
▼
[Route Controller (Business Logic)]
| (If throw new Error() or next(err))
▼ Express skips ALL normal middlewares and jumps directly to:
[Global Error Handler: (err, req, res, next)] → res.status(500).json(...)
```

### ⚠️ 4-Argument Rule

Error middleware me **chaar arguments** (`err`, `req`, `res`, `next`) hona zaroori hai. Agar 3 likhoge toh Express usse regular middleware maan lega!

### 🔄 next() vs return

`next()` ke baad code execute na ho iske liye hamesha `return next()` ya `return res.json()` likhein.



## 2. Complete Middleware Production Pipeline Demo

chapter06\_master\_demo.js

Middleware Pipeline Server

```
const express = require('express');
const app = express();

app.use(express.json());

// Custom Request Timing Middleware:
app.use((req, res, next) => {
  req.requestTime = Date.now();
  next();
});

// Error Trigger Route for Testing:
app.get('/broken-route', (req, res, next) => {
  const error = new Error("Database connection timed out!");
  error.status = 503;
  next(error); // Jumps straight to 4-argument global error handler!
});

// Global 4-Arg Error Handler:
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({
    status: 'error',
    message: err.message,
    timestamp: req.requestTime
  });
});

app.listen(4000);
```

| Middleware Type   | Declaration Style                                              | Primary Purpose                                   |
|-------------------|----------------------------------------------------------------|---------------------------------------------------|
| Application-level | <code>app.use((req, res, next) =&gt; {})</code>                | Global logging, rate limiting, request timing     |
| Router-level      | <code>router.use( ... )</code>                                 | Route group specific guards (e.g. /admin routes)  |
| Built-in          | <code>express.json()</code> ,<br><code>express.static()</code> | Payload parsing and static asset hosting          |
| Error-handling    | <code>(err, req, res, next)</code>                             | Centralized crash protection and error formatting |

## *Request Data & Body Parsing*

---

Topics Explanation with Indian Examples

## 1. *req.params* vs *req.query* vs *req.body* vs *req.headers*

---

### 🤔 1.1 Problem Kya Thi? (Where Does Client Data Live in HTTP?)

Client se server par data 4 alag-alag darwazo se enter hota hai. Agar aapko ye nahi pata ki kaun sa data kahan se nikalna hai, toh aap empty object `undefined` pakadte rahoge:

1. **Route Parameters (`req.params`)**: URL ke path ka part (e.g. `/users/:id` → `req.params.id`). Specific resource identify karne ke liye.
2. **Query String (`req.query`)**: URL ke baad `?` lagakar key-value pairs (e.g. `/search?keyword=node&page=2` → `req.query.keyword`). Filtering, sorting, and pagination ke liye.
3. **Request Body (`req.body`)**: POST/PUT request ka hidden payload (JSON ya Form Data). Iske liye `express.json()` ya `express.urlencoded()` lagana mandatory hai!
4. **Request Headers (`req.headers`)**: Metadata jaise `authorization`, `user-agent`, `content-type`.

## snippet\_request\_extraction.js Extracting Params, Query, Body & Headers

```
const express = require('express');
const app = express();

app.use(express.json()); // ➡ Required for req.body (JSON)
app.use(express.urlencoded({ extended: true })); // ➡ Required for HTML Form submit

// 1. req.params: Dynamic Route Parameters
app.get('/users/:userId/orders/:orderId', (req, res) => {
  const { userId, orderId } = req.params; // Extract "101" and "8823"
  res.json({ userId, orderId });
});

// 2. req.query: Filter & Pagination Query String
app.get('/products', (req, res) => {
  const { category = 'all', sort = 'asc', limit = 10 } = req.query;
  res.json({ category, sort, limit: Number(limit) });
});

// 3. req.body: POST Payload Extraction
app.post('/api/register', (req, res) => {
  const { username, email, password } = req.body;
  res.status(201).json({ message: 'User Registered', username, email });
});
```



### *Indian Real-Life Analogy: Speed Post Parcel Anatomy*

- **req.params (Ghar Ka Flat No):** Address line me likha flat no /tower-a/:flatNo – exact location identify karta hai.
- **req.query (Delivery Instructions):** Envelope ke upar stamp ? deliverBy=evening&callBefore=yes – delivery customize karta hai.
- **req.body (Box Ke Andar Ka Gift):** Sealed box ke andar ka actual saman (Hidden secure payload).
- **req.headers (Sender Post Office Stamp):** Stamp aur bar-code jo prove karta hai parcel authentic hai (Auth Token).



# Visual Architecture Blueprint: HTTP Request Anatomy & Parsing Map

DATA  
INGESTION  
PIPELINE

HTTP Request: **GET /api/v1/jobs/** **450** **?location=Bangalore&remote=true**

- req.params.id = "450"** (Matched via route /api/v1/jobs/:id)
- req.query = { location: "Bangalore", remote: "true" }** (Parsed after ?)
- req.headers['authorization'] = "Bearer eyJhbGciOi ... "** (Auth token)
- req.body = { ... }** (Extracted via `express.json()` for POST/PUT)

## ⚠ Undefined req.body?

Agar  
`app.use(express.json())`  
routes se pehle declare nahi kiya, toh  
`req.body` hamesha **undefined**  
aayega!



## String vs Number

`req.params`  
aur  
`req.query`  
ki values  
hamesha **String**  
hoti hain. Math  
calculations ke  
liye unhe  
`Number()`  
me convert  
karein!

## 2. Complete Request Extraction Server Demo

chapter07\_master\_demo.js

Request Parsing Server

```
const express = require('express');
const app = express();

app.use(express.json());

// Full endpoint utilizing params, query, and body simultaneously:
app.post('/api/company/:companyId/employees', (req, res) => {
  const companyId = req.params.companyId; // From Path
  const department = req.query.dept || 'Engineering'; // From Query string
  const { name, role, salary } = req.body; // From Body

  res.status(201).json({
    status: 'success',
    employee: { companyId, department, name, role, salary: Number(salary) }
  });
});

app.listen(4000);
```

| Data Container | Where Found in Request              | Parser Required?               | Typical Use Case                      |
|----------------|-------------------------------------|--------------------------------|---------------------------------------|
| req.params     | URL Route pattern<br>( :paramName ) | No (Built into Express router) | Locating specific ID<br>( /jobs/:id ) |
| req.query      | URL Query string ( ?key=val )       | No (Built into Express router) | Searching, sorting, filtering         |
| req.body       | HTTP POST/PUT/PATCH payload         | Yes:<br>express.json()         | Creating or updating records          |
| req.headers    | HTTP Request Header map             | No (Auto lower-cased)          | Auth tokens, user-agent, cookies      |

## *Dynamic Routing & REST API Architecture*

---

Topics Explanation with Indian Examples

## 1. REST API Architecture & The Route Precedence Trap

### 🤔 1.1 Problem Kya Thi? (The Dangerous Route Precedence Bug)

Express routes ko **Top to Bottom** order me evaluate karta hai. Agar aapne dynamic parameterized route `/jobs/:id` ko pehle declare kar diya aur static route `/jobs/new` ko baad me, toh jab koi `/jobs/new` par aayega, Express "new" ko `id` samajh kar database me search karega aur `CastError` phak dega!

**Rule of Thumb:** Hamesha specific static routes (`/jobs/new`, `/jobs/stats`) ko dynamic routes (`/jobs/:id`) se **UPAR** likhein!

snippet\_route\_precedence.js

Correct Route Declaration Order

```
const express = require('express');
const app = express();

// ✅ STEP 1: Specific static routes FIRST:
app.get('/jobs/new', (req, res) => {
  res.json({ message: 'Render Job Creation Form' });
});

app.get('/jobs/stats', (req, res) => {
  res.json({ totalJobs: 120, activeJobs: 95 });
});

// ✅ STEP 2: Parameterized dynamic route AFTER:
app.get('/jobs/:id', (req, res) => {
  const jobId = req.params.id;
  res.json({ message: `Details for job #${jobId}` });
});
```

### 📁 1.2 REST Verbs & Status Code Conventions

REST (Representational State Transfer) standard rules define karta hai:

- GET `/jobs`: Read all resources (200 OK)
- GET `/jobs/:id`: Read single resource (200 OK or 404 Not Found)
- POST `/jobs`: Create new resource (201 Created)



- PUT /jobs/:id: Complete replacement of resource (200 OK)
- PATCH /jobs/:id: Partial update of single field (200 OK)
- DELETE /jobs/:id: Remove resource (204 No Content)



### *Indian Real-Life Analogy: Bank Account Counter*

#### *Operations*

- GET (Passbook Print): Sirf account details read karna (No change in balance).
- POST (Naya Account Open): Naya customer record create karna (201 Created).
- PATCH (Mobile Number Update): Sirf ek field update karna bina pura account badle.
- DELETE (Account Close): Account delete karna (204 No Content).



# Visual Architecture Blueprint: REST API Method & Status Code Matrix

CRUD VERB  
ARCHITECTURE

|                      |                      |                 |
|----------------------|----------------------|-----------------|
| GET /api/jobs        | → Read Collection →  | 200 OK          |
| POST /api/jobs       | → Create Record →    | 201 Created     |
| GET /api/jobs/:id    | → Read Single Item → | 200 OK (or 404) |
| PATCH /api/jobs/:id  | → Partial Update →   | 200 OK          |
| DELETE /api/jobs/:id | → Remove Item →      | 204 No Content  |

## ⚡ Idempotency

GET, PUT,  
DELETE  
idempotent hote  
hain — 10 baar  
chalané par bhi  
state me koi  
unexpected side-  
effect nahi hota.

## 🚫 204 No Content

```
res.status(204).send()  
body include nahi karta. Iske sath  
res.json() bhejna HTTP  
violation hai!
```

## 2. Complete RESTful CRUD API Server Demo

chapter08\_master\_demo.js

Full REST CRUD Server

```
const express = require('express');
const app = express();

app.use(express.json());

let jobs = [
  { id: 1, title: 'Backend Node.js Engineer', company: 'Razorpay', salary: 95000 },
  { id: 2, title: 'Frontend React Developer', company: 'Swiggy', salary: 80000 }
];

// 1. GET All Jobs
app.get('/api/jobs', (req, res) => res.json(jobs));

// 2. GET Single Job by ID
app.get('/api/jobs/:id', (req, res) => {
  const job = jobs.find(j => j.id === Number(req.params.id));
  if (!job) return res.status(404).json({ error: 'Job Not Found' });
  res.json(job);
});

// 3. POST Create Job
app.post('/api/jobs', (req, res) => {
  const { title, company, salary } = req.body;
  const newJob = { id: jobs.length + 1, title, company, salary: Number(salary) };
  jobs.push(newJob);
  res.status(201).json(newJob);
});

// 4. PATCH Update Job
app.patch('/api/jobs/:id', (req, res) => {
  const job = jobs.find(j => j.id === Number(req.params.id));
  if (!job) return res.status(404).json({ error: 'Job Not Found' });
  Object.assign(job, req.body);
  res.json(job);
});

// 5. DELETE Job
app.delete('/api/jobs/:id', (req, res) => {
  jobs = jobs.filter(j => j.id !== Number(req.params.id));
  res.status(204).send();
});

app.listen(4000);
```

| HTTP Verb | Route Pattern | Status Code    | Operation                    |
|-----------|---------------|----------------|------------------------------|
| GET       | /api/jobs     | 200 OK         | Retrieve all job listings    |
| POST      | /api/jobs     | 201 Created    | Create a new job record      |
| GET       | /api/jobs/:id | 200 OK / 404   | Retrieve single job record   |
| PATCH     | /api/jobs/:id | 200 OK         | Partially update job fields  |
| DELETE    | /api/jobs/:id | 204 No Content | Remove job record completely |

### 3. JWT Authentication – The #1 Interview Topic @ 6 LPA

#### 🔴 3.1 Problem: HTTP is Stateless – Server remember nahi karta!

HTTP stateless hai – har request independent hai, server kuch remember nahi karta. Pehle **Sessions** use hote the (server-side memory mein user data store). Lekin jab multiple servers hon (load balancer), session ek server pe hoga, dusra server use nahi jaanta. **JWT = Stateless Authentication** – server mein kuch store nahi karna, token hi sab kuch carry karta hai.

#### 📖 3.2 JWT Kya Hota Hai? (3-Part Structure)

JWT (JSON Web Token) ek compact, self-contained token hai jisme **3 parts** hote hain, . se separate:

jwt\_structure.txt

JWT Token Anatomy

```
// JWT Token Example:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJlMScwZSI6ImFkbWl
// Part 1: HEADER (Base64 encoded)
{ "alg": "HS256", "typ": "JWT" } // Algorithm used for signing

// Part 2: PAYLOAD (Base64 encoded – NOT encrypted, visible!)
{
  "userId": 101,
  "role": "admin",
  "iat": 1698765400, // issued at (timestamp)
  "exp": 1698851800 // expiry timestamp – token auto-invalid ho ja
}

// Part 3: SIGNATURE (HMAC-SHA256 with your secret)
// HMAC_SHA256(base64(header) + '.' + base64(payload), JWT_SECRET)
// ↑ Agar koi payload tamper kare, signature match nahi karega → REJECTE
```

#### 📖 3.3 JWT Authentication Flow – Complete Implementation

```
const jwt = require('jsonwebtoken');
const express = require('express');
require('dotenv').config();
const app = express();
app.use(express.json());

// ===== STEP 1: LOGIN ROUTE – Token Generate karo =====
app.post('/api/login', (req, res) => {
  const { email, password } = req.body;

  // DB se user fetch karke password verify karo (simplified here)
  const user = { id: 101, email, role: 'admin' }; // mock DB result

  if (!user) return res.status(401).json({ error: 'Invalid credentials' });

  // JWT generate karo – payload + secret + options
  const token = jwt.sign(
    { userId: user.id, role: user.role }, // PAYLOAD (public info)
    process.env.JWT_SECRET, // SECRET (apna private)
    { expiresIn: '24h' } // EXPIRY
  );

  res.status(200).json({ message: 'Login successful', token });
  // Client is stores this token (localStorage/cookie)
});

// ===== STEP 2: AUTH MIDDLEWARE – Token Verify karo =====
function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  // Format: "Bearer eyJhbGci..."
  const token = authHeader && authHeader.split(' ')[1]; // "Bearer" ke

  if (!token) return res.status(401).json({ error: 'Token required' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { userId: 101, role: 'admin', iat: ..., e
    next(); // Token valid! Aage jao.
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

// ===== STEP 3: PROTECTED ROUTES – Middleware lagao =====
app.get('/api/profile', authenticateToken, (req, res) => {
  // req.user available hai – middleware ne set kiya!
  res.json({ message: 'Private data', user: req.user });
});

app.delete('/api/admin/user/:id', authenticateToken, (req, res) => {
  if (req.user.role !== 'admin')
    return res.status(403).json({ error: 'Admin only!' });
  res.json({ message: `User ${req.params.id} deleted` });
});
```



### *Beginner Terminology: "Bearer Token" Kya Hota Hai?*

"**Bearer**" ka literal matlab hota hai *"The person who holds/carries this token"*. Jaise cinema hall mein ticket jiske haath mein hoti hai (ticket bearer), guard usko movie dekhne deta hai bina uski identity pooche — waise hi HTTP API mein jiske paas valid signed JWT token hai, server usko authenticated maanta hai.

**Standard Header Syntax:** Authorization: Bearer <jwt\_token\_string>



# JWT Flow Anatomy: Login → Token → Protected Request

FULL  
FLOW

**STEP 1** POST /api/login — Server verifies creds —  
`jwt.sign(payload, secret)` → Token

**STEP 2** Client stores token — Har request mein **Authorization:**  
**Bearer <token>** header bhejta hai

**STEP 3** Middleware — `jwt.verify(token, secret)` — Valid?  
req.user set karo → next()

**STEP 4** Invalid/Expired — **JsonWebTokenError /**  
**TokenExpiredError** — 401 bhejo



## Interview Q

"Session vs JWT kya fark hai?" →  
Session = server-side storage (stateful). JWT = client-side token (stateless). JWT microservices ke liye better!



## Security

JWT payload **encrypted** nahi hota, sirf Base64 encoded! Sensitive data (password) payload mein mat daalo. Secret strong rakho!



## *MVC Architecture Pattern & `express.Router()`*

---

Step-by-Step Architecture Guide with Real-World Analogy

## 1. The Problem: 1,000-Line Monolithic app.js Hell

Beginners aksar galti karte hain ki database models, business logic, route handlers, authentication guards, aur template rendering sab kuch ek hi `app.js` file me 1,000 lines ka spaghetti code likh dete hain. Isse code debug karna namumkin ho jata hai aur team me merge conflicts aate hain.

**MVC Solution:** Har kaam ke liye dedicated file aur folder banana (Separation of Concerns):



### *Indian Real-Life Analogy: Super-Specialty Hospital Departmentation*

- **1. Receptionist (Router):** Patient ka token dekh kar batata hai ki ENT department jaana hai ya Cardiology (URL endpoint matching).
- **2. Specialist Doctor (Controller):** Patient ka checkup karta hai, bimari ka decision leta hai (Business Logic).
- **3. Medical Records Room (Model):** Purani patient files aur database store karta hai (Data Access Layer).
- **4. Discharge Summary Card (View):** Patient ko jo sundar printout milta hai (EJS Template / Client Response).



## Step-by-Step Request Flow: From Browser to Database

STEP-BY-STEP MVC  
LIFECYCLE

1. Client Request → **app.js** (Server Entry Point)
2. Forward to → **routes/jobRoutes.js** (Matches URL `/api/jobs/:id`)
3. Executes → **controllers/jobController.js** (getJobById Logic)
4. Queries → **models/jobModel.js** (Fetches DB Record)
5. Responds → **res.json({ success: true, data: job })** → Client Browser ✓



### Modular Scalability

Naya feature add karna ho (e.g. `/users`), toh sirf `routes/users.js` aur `controllers/userController.js` banana padta hai bina purana code disturb kiye!



### Easy Unit Testing

Controllers ko bina server start kiye directly mock request/response ke sath test kiya ja sakta hai.

## 2. Step-by-Step Code Implementation (Ek-Ek Layer Karke)

### ◆ Step 1: Model Layer — Data Store & Database Access

Sabse pehle decide karo ki data kahan aur kaise store hoga. Model layer direct data se deal karti hai:

models/jobModel.js

Layer 1: Model (Data Store)

```
// In-memory data store (ya real MongoDB Mongoose Model):
let jobsDatabase = [
  { id: 1, title: 'Backend Developer', company: 'Razorpay', salary: '6 LPA' },
  { id: 2, title: 'Node.js Engineer', company: 'Swiggy', salary: '8 LPA' }
];

class JobModel {
  static async findAll() {
    return jobsDatabase;
  }

  static async findById(id) {
    return jobsDatabase.find(j => j.id === parseInt(id));
  }

  static async create(jobData) {
    const newJob = { id: Date.now(), ...jobData };
    jobsDatabase.push(newJob);
    return newJob;
  }
}

module.exports = JobModel;
```

### ◆ Step 2: Controller Layer — Business Logic & Decisions

Controller ka kaam hai incoming request validate karna, Model se data mangwana, aur response return karna:

```
const JobModel = require('../models/jobModel');

// 1. Get All Jobs:
exports.getAllJobs = async (req, res, next) => {
  try {
    const jobs = await JobModel.findAll();
    res.status(200).json({ success: true, count: jobs.length, data: jobs });
  } catch (err) {
    next(err); // Global error handler ko pass karo!
  }
};

// 2. Get Single Job by ID:
exports.getJobById = async (req, res, next) => {
  try {
    const job = await JobModel.findById(req.params.id);
    if (!job) {
      return res.status(404).json({ success: false, message: 'Job not found' });
    }
    res.status(200).json({ success: true, data: job });
  } catch (err) {
    next(err);
  }
};

// 3. Create New Job:
exports.createJob = async (req, res, next) => {
  try {
    const { title, company, salary } = req.body;
    if (!title || !company) {
      return res.status(400).json({ success: false, message: 'Title and company are required' });
    }
    const newJob = await JobModel.create({ title, company, salary });
    res.status(201).json({ success: true, data: newJob });
  } catch (err) {
    next(err);
  }
};
```

### ◆ Step 3: Router Layer — URL Endpoint Mapping

Router ka kaam sirf URL routes ko respective Controller methods se connect karna hai:

```
const express = require('express');
const router = express.Router();
const jobController = require('../controllers/jobController');

// Routes relative to '/api/jobs':
router.get('/', jobController.getAllJobs);
router.get('/:id', jobController.getJobById);
router.post('/', jobController.createJob);

module.exports = router;
```

#### ◆ Step 4: Server Assembly in app.js – Sabko Ek Saath Plug Karna

Main app.js file ab bilkul clean (sirf ~15 lines) rehti hai, jisme modular router mount hota hai:

```
const express = require('express');
const app = express();
const jobRoutes = require('./routes/jobRoutes');

// 1. Built-in body parser middleware:
app.use(express.json());

// 2. Mount modular router under /api/jobs:
app.use('/api/jobs', jobRoutes);

// 3. Centralized 4-parameter error handler:
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    success: false,
    error: err.message || 'Internal Server Error'
  });
});

const PORT = process.env.PORT || 4000;
app.listen(PORT, () => console.log());
```

| <i>MVC Layer</i> | <i>Directory Location</i> | <i>Responsibility</i>                                   |
|------------------|---------------------------|---------------------------------------------------------|
| Router           | /routes/*.js              | URL endpoints ko controllers se map karna               |
| Controller       | /controllers/*.js         | Request body validate karna aur Model se interact karna |
| Model            | /models/*.js              | Database schema, queries, aur data integrity rules      |
| View             | /views/*.ejs              | User Interface HTML generation (SSR)                    |

## *Static Assets & EJS Templating Engine*

---

Topics Explanation with Indian Examples



## 1. Server-Side Rendering (SSR) & EJS Tag Mechanics

### 🤔 1.1 Problem Kya Thi? (Why Plain Static HTML is Not Enough)

Static .html files me dynamic database data (jaise logged-in user ka username ya shopping cart items) inject nahi ho sakta. Agar server-side rendering na ho, toh har user ke liye alag HTML file banani padegi jo namumkin hai!

**EJS Solution:** EJS (Embedded JavaScript) ek templating engine hai jo HTML ke andar JavaScript variables aur loops run karke server par hi final HTML render karta hai.

views/jobs.ejs

EJS Tags Syntax

```
<!-- 1. <%= %> Raw Unescaped: Used for Reusable Partials (Navbar/Footer) -->
<%= include('partials/header') %>

<main class="container">
  <!-- 2. <%= %> Escaped Safe Variable Output: Protected against XSS injection! -->
  <h1>Welcome back, <%= user.username %>!</h1>

  <!-- 3. <% %> Scriptlet Tag: JavaScript Logic (Loops & Conditions) -->
  <% if (jobs.length === 0) { %>
    <p>No job openings available right now.</p>
  <% } else { %>
    <div class="job-grid">
      <% jobs.forEach(function(job) { %>
        <div class="job-card">
          <h3><%= job.title %></h3>
          <p>Company: <strong><%= job.company %></strong></p>
          <span>Salary: ₹<%= job.salary %></span>
        </div>
      <% }); %>
    </div>
  <% } %>
</main>

<%= include('partials/footer') %>
```

| EJS Tag                        | Tag Name          | Kya Kaam Karta Hai? (Plain Hindi)                                                                                                  | Security Note                                                                           |
|--------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>&lt;%= data %&gt;</code> | Escaped Output    | Variables print karta hai (e.g. user name). HTML characters ( <code>&lt;</code> , <code>&gt;</code> ) ko safe text me badalta hai. | ✅ <b>Safe against XSS:</b> Hacker ka <code>&lt;script&gt;</code> inject nahi hone deta! |
| <code>&lt;%= html %&gt;</code> | Unescaped Output  | Raw HTML print karta hai (e.g. header/footer partials include karne ke liye).                                                      | ⚠️ <b>Dangerous with user input:</b> Sirf trusted internal files ke liye use karein.    |
| <code>&lt;% code %&gt;</code>  | Scriptlet (Logic) | JS ka control flow run karta hai (e.g. <code>if/else</code> , <code>forEach</code> loops). Screen par kuch print nahi karta.       | ✅ Pure logic execution block.                                                           |

## 🎨 1.2 Serving Static Assets securely with `express.static()`

CSS files, images (`logo.png`), aur client-side JavaScript ko serve karne ke liye `express.static` built-in middleware use hota hai:

snippet\_static\_files.js

Static Asset Hosting

```
const path = require('path');
const express = require('express');
const app = express();

// Configure EJS View Engine:
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Serve static assets securely from /public folder:
app.use(express.static(path.join(__dirname, 'public')));

app.get('/portal', (req, res) => {
  res.render('jobs', {
    user: { username: 'Aarav' },
    jobs: [{ title: 'Backend Dev', company: 'Google', salary: 120000 }]
  });
});
```



### Indian Real-Life Analogy: Shaadi Ka Printed Invitation Card

- **EJS Template (Blank Gold-Foil Card):** Fixed design, borders, aur venue address jo sabke liye same hai.
- **res.render Data ({ user, jobs }):** Mehmaan ka naam jo printer card ke upar print karke complete card tayyar karta hai!



# Visual Architecture Blueprint: EJS

## Server-Side Compilation Flow

SERVER-SIDE  
RENDERING

```
[ jobs.ejs Template ] + [ Database Object: { jobs: [ ... ] } ]
```

▼ **EJS Compiler Engine** (Evaluates loops, injects variables)

▼ **Generates Plain Static HTML String**

▼ **res.send(finalHtml)** —> Sets Content-Type: text/html —> Browser renders instantly! 🚀



### XSS Prevention

`<%= %>` automatic  
HTML entities escape  
karta hai (`<script>`  
becomes  
`&lt;script&gt;`)  
taaki hackers malicious  
script run na kar  
sakein!



### Partials

`<%-`  
`include('partials/header')`  
`%>` se Navbar aur Footer ko 1 hi jagah  
maintain kiya jata hai.

## 2. Complete EJS Portal Server Demo

chapter10\_master\_demo.js

EJS Full Server

```
const path = require('path');
const express = require('express');
const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(express.static(path.join(__dirname, 'public')));

app.get('/', (req, res) => {
  const mockJobs = [
    { title: 'Node.js Architect', company: 'Microsoft', salary: 180000 },
    { title: 'Fullstack Lead', company: 'Amazon', salary: 160000 }
  ];
  res.render('jobs', { user: { username: 'Rahul' }, jobs: mockJobs });
});

app.listen(4000);
```

| EJS Tag        | Syntax                                   | Behavior                              | Security                      |
|----------------|------------------------------------------|---------------------------------------|-------------------------------|
| Escaped Output | <code>&lt;%= variable %&gt;</code>       | Prints text safely                    | Protected against XSS         |
| Raw Output     | <code>&lt;%- include( ... ) %&gt;</code> | Renders raw HTML string               | Use only for trusted partials |
| Scriptlet      | <code>&lt;% for/if %&gt;</code>          | Executes control logic without output | Zero output leakage           |

## *Production-Ready Node.js – cors, helmet, Error Handling & nodemon*

---

Jo sirf ye jaanta hai wo 6 LPA se 10 LPA jaata hai!

## 1. cors – Cross-Origin Resource Sharing

### 1.1 Problem: Browser ne request block kar di – CORS Error!

Jab frontend (localhost:3000) backend (localhost:5000) ko request bhejta hai – alag port = alag "origin". Browser security policy (Same Origin Policy) isko block kar deta hai. Ye error sirf browser mein aata hai – Postman/curl mein nahi! CORS package server ko batata hai ki kaunse origins ko allow karna hai.

`cors_setup.js`

Production CORS Configuration

```
const cors = require('cors');

// ❌ DON'T: Open to everyone (use only during dev!)
app.use(cors()); // Any origin can access – security risk!

// ✅ DO: Restrict to specific origins in production
const corsOptions = {
  origin: ['https://myjobapp.vercel.app', 'http://localhost:3000'],
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true // Allow cookies/auth headers across origins
};
app.use(cors(corsOptions));

// CORS headers jo server response mein add hote hain:
// Access-Control-Allow-Origin: https://myjobapp.vercel.app
// Access-Control-Allow-Methods: GET, POST, PUT...
// Browser ye headers check karta hai – allowed ho toh proceed karta hai!
```



## CORS Error vs No CORS Error

DEBUG TIP

**CORS Error in Browser:** "Access blocked by CORS policy" → Server ne origin allow nahi kiya

**No Error in Postman:** Postman browser nahi hai, Same Origin Policy nahi follow karta

**Fix:** Backend pe cors() middleware lagao with correct origins



### Interview Q

"CORS kya hai?" → Browser security feature. Different origins ke beech requests control karta hai. Server headers set karta hai jo browser check karta hai.



### Preflight

Browser pehle OPTIONS request bhejta hai (preflight) — check karta hai ki actual request allowed hai ya nahi. Ye automatic hota hai!

## 2. helmet – Security Headers (Free Armor)

### 📖 2.1 helmet Kya Karta Hai?

helmet Express middleware hai jo HTTP security headers automatically set karta hai. Ye headers browser ko different attacks se bachata hai jaise XSS, clickjacking, MIME sniffing. Ek line lagao – 15+ vulnerabilities cover!

helmet\_setup.js

Security Headers

```
const helmet = require('helmet');

// Ek hi line – 15+ security headers set ho jaate hain!
app.use(helmet());

// helmet ye headers set karta hai (examples):
// X-Content-Type-Options: nosniff      → MIME type sniffing rokta hai
// X-Frame-Options: DENY                → Clickjacking rokta hai
// X-XSS-Protection: 1; mode=block     → XSS filter enable
// Strict-Transport-Security            → HTTPS enforce karta hai
// Content-Security-Policy              → Script injection rokta hai

// Custom configuration (agar kuch disable karna ho):
app.use(helmet({
  contentSecurityPolicy: false // API server ke liye turn off kar sakte ho
}));

// ✅ Production mein ye order follow karo:
app.use(helmet());           // 1. Security headers
app.use(cors(corsOptions));  // 2. CORS
app.use(express.json());     // 3. Body parser
app.use('/api', routes);     // 4. Routes
app.use(errorHandler);       // 5. Global error handler (last!)
```



### 3. Global Error Handler Middleware – Ek Central Error Catch

#### ● 3.1 Problem: Har route mein try/catch = code duplication nightmare!

Agar har route mein alag try/catch aur alag error response likho – 50 routes mein 50 baar same error handling code. DRY principle violate!  
Solution: Ek **global error handler middleware** jo sabki errors pakad le.

errorHandler.js + app.js

Global Error Handling Pattern

```
// ===== routes/jobs.js – Sirf error throw karo, handle mat karo! =====
const asyncHandler = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

router.get('/:id', asyncHandler(async (req, res) => {
  const job = await Job.findById(req.params.id);
  if (!job) {
    const err = new Error('Job not found');
    err.statusCode = 404;
    throw err; // asyncHandler catches this and calls next(err)
  }
  res.json(job);
}));

// ===== middleware/errorHandler.js =====
function errorHandler(err, req, res, next) { // 4 params = error middleware!
  console.error(`[ERROR] ${err.message}`, { url: req.url, method: req.method });

  const statusCode = err.statusCode || 500;
  const message = err.message || 'Internal Server Error';

  res.status(statusCode).json({
    success: false,
    error: message,
    // Stack trace only in dev – never in production!
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
  });
}
module.exports = errorHandler;

// ===== app.js – LAST middleware! =====
app.use(errorHandler()); // Must be AFTER all routes!
```



## Anatomy: Error Middleware – Why 4 Parameters?

CRITICAL  
RULE

Regular middleware: **(req, res, next)** → 3 params

**Error middleware: (err, req, res, next)** → 4 params – Express isko error handler maan leta hai!

Jab bhi **next(err)** call karo ya async error throw ho – Express automatically is middleware pe jaata hai.



### Must be LAST

Error handler middleware hamesha sab routes ke BAAD register karo. Pehle lagaoge toh routes ki errors nahi pakad paayega!



### Interview Q

"Express error middleware regular middleware se kaise alag hai?" → 4 parameters! (err, req, res, next) – pehla parameter err hota hai.

#### 4. nodemon – Auto-Restart Dev Server & File Watcher Internals

##### 🔴 4.1 Problem Kya Thi? (The 100-Times Manual Restart Fatigue)

Standard Node.js mein jab aap `node app.js` chalte ho, toh Node memory mein pura code load karke run kar deta hai. Agar aap file mein 1 single line ya spelling bhi change karoge, toh running server ko pata nahi chalta. Developer ko manually:

1. Terminal mein jaakar `Ctrl + C` daba kar server kill karna padta hai.
2. Phir se `node app.js` command type karke restart karna padta hai.
3. Din bhar mein 200 baar ye repeat karna developer ki productivity barbad kar deta hai!

#### ⚡ *Indian Real-Life Analogy: Purana Kerosene Generator vs Smart Auto-Inverter*

- **Manual Node (`node app.js`):** Purana generator jisme light aane-jaane par har baar bahar jaakar rassi kheench ke manually start karna padta hai.
- **Nodemon:** Smart automatic inverter — jaise hi room mein koi switch on/off hota hai (file save hoti hai), ye 0.1 second mein bina kisi manual effort ke restart karke fresh state deliver kar deta hai.



# Visual Anatomy: Nodemon Process Lifecycle & Auto-Reload Engine

FILE WATCHER  
INTERNALS

1. Developer edits code → **Ctrl + S (File Saved)**
2. File System Event → **chokidar / fs.watch()** detects modified timestamp
3. Kill Signal → **SIGUSR2 / SIGTERM** sent to existing child Node.js PID (Port released)
4. Spawn Fresh Process → **child\_process.spawn('node app.js')** executes new code
5. Ready in 100ms → **[nodemon] restarting server ...** **Live on Port 4000**



## Never in Production!

Nodemon ko sirf **development** mein use karein (`--save-dev`).  
Production (AWS/Docker/Kubernetes) mein standard `node app.js` ya PM2 process manager chalta hai!



## node\_modules Loop Trap

Agar nodemon ko `node_modules/` ignore karne ko na bolo, toh koi naya package install hone par server infinite loop mein restart hota rahega!



## 4.2 Step-by-Step Installation & NPM Scripts Setup

**Step 1: Install as Developer Dependency** (Production bundle mein add nahi hona chahiye):

### Terminal

### Dev-Dependency Installation

```
# -D / --save-dev flag nodemon ko package.json ke devDependencies section mein daalt.  
npm install --save-dev nodemon
```

**Step 2: package.json Scripts Configure Karna:**

## package.json

## Production vs Development Scripts

```
{
  "name": "techjobs-api",
  "scripts": {
    "start": "node app.js", // 🚀 Production standard (AWS/Docker/Render)
    "dev": "nodemon app.js", // 🚀 Development auto-restart: npm run dev
    "dev:debug": "nodemon --inspect app.js" // 🚀 Chrome DevTools Inspector attach k
  }
}
```

### 🔧 4.3 Advanced Project Configuration with nodemon.json

Bade enterprise projects mein hum root directory mein

nodemon.json config file banate hain taaki exact rules set kar sakein:

## nodemon.json

## Fine-Grained Watch Rules

```
{
  "watch": ["src/", "controllers/", "routes/"], // 1. Sirf in folders ke changes tra
  "ext": "js,json,.env,ejs", // 2. In file extensions par relo
  "ignore": ["node_modules/", "logs/", "*.test.js"], // 3. In folders aur test files
  "delay": 500, // 4. 500ms debounce buffer (ra
  "env": {
    "NODE_ENV": "development"
  }
}
```

nodemon.json  
Key

Type

Kya Kaam Karta Hai? (Plain Hindi)

Why Critical?

watch

Array of Strings

Specific folders define karta hai jinhe monitor karna hai

Faltu background folders scan hone se CPU usage bachta hai

ext

Comma-separated String

Kin extensions (e.g. `.js`, `.env`, `.ejs`) par reload karna hai

HTML/CSS/ENV changes par bhi server auto-refresh hota hai

ignore

Array of Strings

Kin directories ko monitor nahi karna

`node_modules` aur `logs` folder se infinite loop rokta hai

delay

Number (ms)

File save hone ke baad kitne millisecond ruk kar restart karna hai

Debouncing — jab aap 5 files ek sath save karte ho toh 5 baar restart nahi hota

## 5. Complete Production app.js Template – Sab Saath

✅ Industry-Standard Express.js Starter – Har Interview/Job Mein Use Hota Hai

app.js

Production-Ready Express Template

```
// 1. ENV – MUST be first!
require('dotenv').config();

const express      = require('express');
const cors          = require('cors');
const helmet        = require('helmet');
const jobRoutes     = require('./routes/jobs');
const errorHandler = require('./middleware/errorHandler');

const app = express();

// ===== GLOBAL MIDDLEWARE (ORDER MATTERS!) =====
app.use(helmet()); // 1. Security headers
app.use(cors({ origin: process.env.CLIENT_URL })); // 2. CORS
app.use(express.json({ limit: '10kb' })); // 3. Body parser (limit size!)

// ===== ROUTES =====
app.use('/api/jobs', jobRoutes);
app.use('/api/auth', authRoutes);

// ===== 404 Handler =====
app.use((req, res) => {
  res.status(404).json({ error: `Route ${req.method} ${req.url} not found` });
});

// ===== GLOBAL ERROR HANDLER (LAST!) =====
app.use(errorHandler);

// ===== START SERVER =====
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`🚀 Server live on port ${PORT} [${process.env.NODE_ENV}]`);
});
```



## Middleware Order – Kyu Important Hai?

CRITICAL

1. **helmet()** — Security headers → Har response pe lagta hai
2. **cors()** — Browser ke preflight OPTIONS requests handle karta hai
3. **express.json()** — req.body parse karta hai – PEHLE routes se!
4. **Routes** — Business logic
5. **errorHandler** — LAST! Sab routes ke errors pakad ta hai



### Production Checklist

dotenv ✓ helmet ✓  
cors (specific  
origins) ✓  
express.json() ✓  
Global error  
handler ✓ .env in  
.gitignore ✓



### Interview Q

"Express mein middleware  
order kyun important hai?" →  
Request top-to-bottom flow  
karta hai. Galat order =  
bug/security hole!



## Curated Question Bank

### 30+ Node.js & ES6 Technical Interview Masterclass

Extracted directly from top product company interviews and ES6 syllabus. Filter by category below!

Q 01

ES6 • TDZ

*What is Temporal Dead Zone (TDZ) and why did ES6 introduce it?*

Ans: TDZ is the period between entering block scope and variable declaration. Unlike `var` which initialized to `undefined` (causing silent bugs), `let` and `const` stay uninitialized. Accessing them inside TDZ throws a runtime `ReferenceError`.

Q 02

ES6 • Functions

*Why can't Arrow Functions be used as Object methods or Constructors?*

Ans: Arrow functions do not bind their own dynamic `this` (they lexically inherit `this` from outer scope) and lack internal `[[Construct]]` method and `.prototype` property. Calling `new ArrowFunc()` throws a `TypeError`.

Q 03

ES6 • Microtasks

*What is the difference between Microtask Queue and Macrotask Queue?*

Ans: Microtask Queue has strict higher priority. `Promise.then()`, `Promise.catch()`, and `process.nextTick()` run immediately after current synchronous script completes, before the engine picks next Macrotask (such as `setTimeout`).

Q 04

ES6 • Memory

*Shallow Copy vs Deep Copy in JavaScript – how do they differ?*

Ans: Spread `{ ...obj }` creates a **shallow copy** (top-level values cloned, nested references shared). For a **deep copy**, use `structuredClone(obj)` or `JSON.parse(JSON.stringify(obj))`.



Q 05

ES6 • Closures

*What is a Closure and where is it used in Node.js backend architecture?*

Ans: A closure is a function bundled with references to its surrounding lexical scope. Used in Node for data encapsulation, middleware factories (e.g. `express.urlencoded()` returning configured middleware), and memoization caches.

Q 06

Node • Concurrency

*If Node.js is single-threaded, how does it handle thousands of concurrent requests?*

Ans: JS execution stack is single-threaded, but Node offloads blocking I/O (file system, DNS, crypto) to the **libuv C threadpool** (default 4 threads) or OS kernel `epoll/kqueue`. When I/O completes, libuv pushes callback to event loop queue without blocking main JS thread.

Q 07

Node • Event Loop

*List the 6 phases of the Node.js Event Loop in their exact order.*

Ans: 1. Timers → 2. Pending Callbacks → 3. Idle/Prepare → 4. Poll → 5. Check → 6. Close Callbacks.

Q 08

Node • Microtasks

*What happens if `process.nextTick()` is called recursively in Node.js?*

Ans: Causes **I/O Starvation**. Because `nextTick` queue is drained completely before Event Loop can move to next phase, recursive `nextTicks` trap engine in infinite loop, blocking Timers and I/O!

Q 09

FS • Safety

*Why is `fs.readFileSync()` forbidden in high-concurrency production servers?*

Ans: Because JS runs on single main thread. `readFileSync` halts entire thread until hard drive finishes reading, freezing every other incoming user request.

Q 10

Streams • Flow

*What is Stream Backpressure and how does `.pipe()` resolve it?*

Ans: Backpressure happens when readable stream emits data faster than writable stream can write to disk. `.pipe()` pauses readable stream when internal buffer is full and resumes when `drain` event fires.

Q 11

Express • Error Handler

*Why must an error-handling middleware explicitly define exactly 4 parameters `(err, req, res, next)`?*

Ans: Express checks handler arity via `fn.length`. If `fn.length === 4`, Express registers it as an Error Middleware and only invokes it when `next(err)` is called or an error throws.

Q 12

Routing • Precedence

*Why must `/jobs/new` be placed before `/jobs/:id` in route definitions?*

Ans: Express matches routes from top to bottom. If `/:id` is registered first, string `"new"` will be matched as the `id` param, invoking the wrong handler.

Q 13

NPM • Lockfile

*Why must `package-lock.json` be committed to Git repositories?*

Ans: `package.json` allows semver ranges (like `^5.0.0`), which means two developers running `npm install` on different days might get different minor/patch versions. `package-lock.json` pins the exact dependency tree down to the exact commit and cryptographic hash, ensuring 100% reproducible builds across all machines and production servers.

Q 14

Node • Modules

*What is the key difference between CommonJS (`require`) and ES Modules (`import`)?*

Ans: CommonJS ( `require()` / `module.exports` ) loads synchronously at runtime (can be used inside if-blocks). ES Modules ( `import` / `export` ) are parsed and evaluated asynchronously and statically before code executes, enabling dead-code elimination (tree-shaking).

Q 15

REST • HTTP Methods

*What is the core difference between `PUT` and `PATCH` HTTP methods?*

Ans: `PUT` replaces the *entire* resource entity (if a field is omitted in the payload, it will be wiped or overwritten with null/defaults). `PATCH` performs a *partial update*, modifying only the specific fields provided in the request body while leaving other fields untouched.

Q 16

HTTP • Status Codes

*What is the difference between 401 Unauthorized and 403 Forbidden?*

Ans: **401 Unauthorized** = Authentication failure (the server doesn't know who you are; invalid/missing token). **403 Forbidden** = Authorization failure (the server knows who you are, but you don't have permission/roles to access this resource).

Q 17

Express • Body Parsing

*What does `express.json()` do under the hood, and what happens if you forget it?*

Ans: Under the hood, it intercepts incoming request streams with `Content-Type: application/json`, buffers the incoming byte chunks, parses the raw JSON string via `JSON.parse()`, and mounts the resulting JS object onto `req.body`. If omitted, `req.body` remains `undefined`.

Q 18

Express • Data Extraction

*When should you use `req.params` vs `req.query` vs `req.body`?*

Ans: `req.params` identifies a *specific resource* in the URL path (e.g. `/jobs/:id`). `req.query` is for *filtering, sorting, and pagination* via optional key-value URL query strings (e.g. `/jobs?role=dev&page=2`). `req.body` holds large or sensitive payloads sent in POST/PUT/PATCH requests (e.g. user registrations, form submissions).

Q 19

Express • Middleware

*What happens if a custom middleware neither calls `next()` nor sends a response?*

Ans: The HTTP request pipeline hangs indefinitely in pending state. The client browser or API consumer will wait until its internal timeout threshold expires (e.g. 2 minutes) and then throw a 504 Gateway Timeout or `ERR_EMPTY_RESPONSE`.

Q 20

Security • JWT

*What is a JWT (JSON Web Token) and what are its 3 parts?*

Ans: A JWT is a compact, URL-safe, stateless token for securely transmitting claims between parties. Its 3 dot-separated base64url-encoded parts are: **Header** (algorithm & token type), **Payload** (user claims like `userId`, `role`, `expiration`), and **Signature** (hash of Header + Payload signed with a secret key to prevent tampering).

Q 21

Security • Auth Headers

*Where and how should a JWT token be transmitted from client to server?*

Ans: In the HTTP `Authorization` request header using the standard Bearer scheme: `Authorization: Bearer <token>`. On the server, middleware extracts it with `req.headers.authorization?.split(' ')[1]` and verifies it via `jwt.verify(token, SECRET)`.

Q 22

Architecture • MVC

*What is the MVC Architecture pattern and why is it preferred over giant route files?*

Ans: MVC enforces Separation of Concerns: **Model** manages raw data and database schemas, **View** handles UI templates / presentation (or client JSON), and **Controller** contains the core business logic. It prevents bloated 1000-line `app.js` files, enables independent unit testing, and allows team collaboration without merge conflicts.

Q 23

SSR • EJS Tags

*What is the difference between `<%= %>`, `<%- %>`, and `<% %>` in EJS?*

Ans: `<%= val %>` outputs HTML-escaped text (safe against XSS attacks). `<%- val %>` outputs raw, unescaped HTML. `<% jsCode %>` executes pure control-flow JavaScript (loops, if-conditions) without outputting any text to the template.

Q 24

Security • CORS

*What is CORS and why do browsers block requests by default?*

Ans: Cross-Origin Resource Sharing is a browser security mechanism based on the Same-Origin Policy. It prevents malicious websites (e.g. evil.com) from making unauthorized AJAX requests to your backend API (api.bank.com) using a user's saved cookies/credentials. Express uses the `cors` package to set `Access-Control-Allow-Origin` headers allowing trusted origins.

Q 25

Security • Helmet

*Why is 'helmet' middleware considered mandatory for production Express apps?*

Ans: `helmet()` automatically sets 15+ HTTP security headers, including hiding `X-Powered-By: Express` (preventing attackers from identifying server technology), enforcing `Strict-Transport-Security` (HSTS), and blocking XSS with `X-Content-Type-Options: nosniff`.

Q 26

Production • Env Security

*Why must '.env' files NEVER be committed to Git and how does 'dotenv' load them?*

Ans: `.env` stores high-security secrets (database passwords, JWT secret keys, API credentials). If committed to GitHub, bots scan and exploit them immediately. `dotenv.config()` reads the local `.env` file on startup and injects key-value pairs into Node's global `process.env` object.

Q 27

Express • Error Handling

*How does a Global Error Handler work and why must it be registered LAST in Express?*

Ans: When any route or middleware passes an error via `next(err)` or throws in synchronous code, Express skips all standard middleware and jumps directly to the 4-parameter error handler `(err, req, res, next)`. It must be registered last because Express executes middleware in strict linear registration order.

Q 28

ES6 • Higher Order Functions

*How do 'map', 'filter', and 'reduce' differ in JavaScript array transformations?*

Ans: `map()` transforms every element 1:1 and returns a new array of the same length. `filter()` selects elements matching a boolean predicate and returns a subset array. `reduce()` folds/aggregates all array elements into a single accumulated output value (number, object, array). None of them mutate the original array.

Q 29

Node • Streams vs Buffer

*Why should you use 'fs.createReadStream()' instead of 'fs.readFile()' for files > 100MB?*

Ans: `fs.readFile()` loads the entire file into RAM at once. If 50 users request a 200MB file simultaneously, memory spikes to 10GB, crashing the Node.js process with an `Out of Memory (OOM)` fatal error. `fs.createReadStream()` reads the file in tiny 64KB chunks and streams them on the fly, keeping memory usage constant at ~64KB.

Q 30

Express • REST Best Practice

*Why is 'res.status(201).json({ success: true, data })' standard practice in REST APIs?*

Ans: It provides explicit HTTP status code semantics for API clients (201 Created vs default 200 OK) while returning a predictable, consistent envelope format (`success` flag, `data` payload, optional `message`) that frontend clients and mobile apps can reliably parse.